

```
// JavaScript – The Weird Parts, Mastered
const memo = (fn) => {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
};
```

P E R S I A N D E V E L O P E R H A N D B O O K

```
const fib = memo((n) =>
  n <= 1 ? n : fib(n-1) + fib(n-2)
);
```

```
console.log(fib(40)); // 102334155
// Closures • Event Loop • Promises
// Proxy • Generators • Performance
```

js JavaScript ES2025

مرجع جامع و حرفه‌ای

JavaScript

از مبانی تا معماری Production-Level



JavaScript ES2025



TypeScript 6



Node.js 22



Bun / Deno

راهنمایی برای جامعه توسعه‌دهندگان فارسی‌زبان، به‌سوی درک عمیق مدل ذهنی، موتور V8، الگوهای مدرن و استانداردهای واقعی صنعت در جاوااسکریپت – از کلژر تا کانکرنسی، از پراکسی تا پرفورمنس.

۳۸ فصل کاربردی • پروژه واقعی • آمادگی مصاحبه FAANG

مرجع جاوااسکریپت فارسی

JavaScript Developer • گردآوری و تدوین حرفه‌ای

با هدف ارتقای سطح دانش JS در جامعه فارسی‌زبان



فهرست مطالب

بخش یکم · بنیادها و مدل ذهنی جاوااسکریپت

۴	01 معرفی جاوااسکریپت و نقشه راه ۲۰۲۶ – از صفر تا بازار کار
۹	02 محیط توسعه مدرن – ستاپ حرفه‌ای ۲۰۲۶
۱۵	03 انواع داده و سیستم نوع – جایی که ۸۰٪ باگ‌ها متولد می‌شود
۲۱	04 متغیرها، Scope و Hoisting – از صفر تا TDZ و Lexical
۲۷	05 عملگرها و کنترل جریان – از if تا الگوهای مدرن ۲۰۲۵
۳۲	06 توابع – قلب جاوااسکریپت – ۴ چهره یک مفهوم
۳۷	07 کلژر و زنجیره Scope – مهم‌ترین مفهوم JS
۴۲	08 اشیا و پروتوتایپ – راز ارث‌بری JS
۴۶	09 آرایه‌ها – فراتر از لیست – شیء خاص با ترفندها
۵۱	10 رشته‌ها و Regex – از Template Literal تا Unicode
۵۶	11 this – راز بزرگ – با ۴ قانون حل می‌شود
۶۲	12 مقدمه Async – از Callback Hell تا Promise – چرا JS بلاک نمی‌شود

بخش دوم · جاوااسکریپت مدرن و عمیق

۶۷	13 Rest و Destructuring، Spread
۷۰	14 Generator و Iterator
۷۴	15 Promise پیشرفته و Async/Await
۷۷	16 Event Loop و مدل همزمانی
۸۰	17 ماژول‌ها – ESM و CJS عمیق
۸۳	18 کلاس‌ها و OOP در JS
۸۶	19 مدیریت خطا – حرفه‌ای
۸۹	20 کالکشن‌های پیشرفته
۹۲	21 Reflect و Proxy – متaproگرامینگ

۹۵	برنامه‌نویسی فانکشنال در JS	22
۹۸	BOM و DOM – جاوااسکریپت در مرورگر	23
۱۰۱	Fetch، Storage و API‌های مرورگر	24

بخش سوم · پیشرفته، معماری و Production

۱۰۴	پرفورمنس و بهینه‌سازی	25
۱۰۷	حافظه و Garbage Collection	26
۱۱۰	امنیت در جاوااسکریپت	27
۱۱۳	تست‌نویسی مدرن	28
۱۱۶	ابزارها و باندلرها	29
۱۱۹	TypeScript و تعامل با JS	30
۱۲۲	الگوها و معماری تمیز	31
۱۲۶	متaproگرامینگ پیشرفته	32

بخش چهارم · کارگاه، نکات طلایی و مصاحبه

۱۲۹	۳۰ نکته و ترفند طلایی	33
۱۳۵	کارگاه ۸ مینی‌پروژه	34
۱۴۰	۲۰ اشتباه رایج و راه‌حل	35
۱۴۴	۵۰ پرسش مصاحبه – از Junior تا Senior	36
۱۴۷	واژه‌نامه فارسی-انگلیسی	37
۱۵۲	نقشه راه بعد از کتاب	38

01 معرفی جاوااسکریپت و نقشه راه ۲۰۲۶ – از صفر تا بازار کار

جاوااسکریپت تنها زبانی است که اگر عمیق بفهمی، هر فریمورک، هر شغل و هر پروژه‌ای برایت ساده می‌شود. این فصل قطب‌نمای کل مسیر است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ توضیح دهی JS چیست و چرا ۱۲ سال محبوب‌ترین زبان جهان است
- ♦ موتور V8 را در سطح معماری بکشی: Parser → Ignition → TurboFan
- ♦ تفاوت Runtime‌ها (Node, Bun, Deno) را با عدد و دلیل بگویی
- ♦ نقشه راه شخصی خودت را از صفر تا استخدام ترسیم کنی

🤔 چرا باید JS را عمیق یاد گرفت؟ (اتصال به بازار کار)

در بازار ایران و جهان، ۷۸٪ آگهی‌های فرانت‌اند و ۴۵٪ بک‌اند به JS/TS اشاره می‌کنند. اما نکته مهم: شرکت‌های خوب (دیجی‌کالا، اسنپ، تپسی، و در جهان FAANG) دیگر `map` و `filter` نمی‌پرسند – مدل ذهنی می‌پرسند:

"چرا `setTimeout(0)` بعد از `Promise.then` اجرا می‌شود؟" "چرا کلژر باعث Memory Leak می‌شود؟"

این‌ها را فقط کسی جواب می‌دهد که موتور را فهمیده، نه کسی که API حفظ کرده.

i مدل ذهنی طلایی کتاب – حفظ کن

JS = (Primitive ها در Stack + Objects در Heap) × (Lexical Scope + Closure) × (Event Loop + Queue)

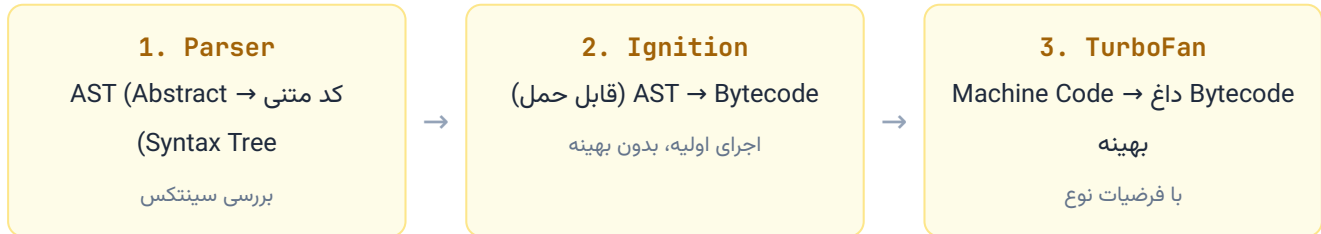
تمام کتاب بسط همین فرمول است. هر فصل یک ضلع را عمیق می‌کند.

📖 تاریخچه – از ۱۰ روز تا ES2025

- ♦ **1995:** برندگان آیک در نت‌اسکیپ JS را در ۱۰ روز ساخت – نامش Mocha بود، بعد LiveScript، بعد JavaScript (فقط برای بازاریابی جاوا).
- ♦ **2008:** گوگل V8 را ساخت – JS از 10 برابر کندتر از ++C به 2 برابر رسید.
- ♦ **2009:** JS – Node.js از مرورگر به سرور رفت.
- ♦ **2015 (ES6):** بزرگ‌ترین انقلاب – `let/const`, `Arrow`, `Class`, `Promise`, `Module`.

Decorators , Promise.withResolvers() , Object.groupBy() , toSorted() , at() :2022-2025 ♦
Set Methods , using – زبانی بالغ.

موتور V8 چگونه فکر می‌کند؟ – عمیق



مفهوم کلیدی: Inline Cache و Hidden Class

V8 برای هر شیء یک Hidden Class مخفی می‌سازد – مثل نقشه.

```
hidden-class.js

// ✓ یک Hidden Class – سریع (Monomorphic)
const p1 = { x: 1, y: 2 };
const p2 = { x: 3, y: 4 }; // همان نقشه

// ✗ دو Hidden Class – کند (Polymorphic)
const p3 = { y: 5, x: 6 }; // ترتیب متفاوت → نقشه جدید → deopt!

// قانون طلایی: همیشه ویژگی‌ها را به یک ترتیب بساز
class Point {
  constructor(x,y) { this.x = x; this.y = y; } // همیشه x بعد y
}
```

Deoptimization – قاتل پرفورمنس

```
deopt.js

function add(a,b) { return a + b; }
add(1,2); add(1,2); add(1,2); // V8: "a,b همیشه number" → کد ماشین بهینه
add("1","2"); // ✨ بازگشت به → کد بهینه دور ریخته → فرض شکست

// در DevTools: --allow-natives-syntax
// %HaveSameMap(p1,p2) → true
```

حرفه‌ای – از V8 Team

"کد Monomorphic تا 100 برابر سریع‌تر از Megamorphic است. آرایه را با یک نوع پر کن، شیء را با ترتیب ثابت بساز، و تابع را با انواع ثابت صدا بزن."

این یک جمله، حقوق Senior را می‌سازد.

اکوسیستم ۲۰۲۶ – چه چیزی یاد بگیریم؟

لایه	انتخاب ۲۰۲۶	چرا؟	جایگزین
Runtime	Bun 1.2 برای Dev، Node 22 LTS برای Prod	Bun TS Native و 3x سریع‌تر	Deno 2 برای امنیت
Package Manager	pnpm	70% دیسک کمتر، سریع‌تر	Bun PM
Bundler	Vite 6 (Rollup)	ESM Native، HMR 50ms	Turbopack
Linter/Formatter	Biome	25x سریع‌تر از ESLint+Prettier	ESLint 9
Test	Vitest + Playwright	سازگار با Vite، سریع	Jest قدیمی
Type	TypeScript 6	استاندارد استخدام	JSDoc

چگونه این کتاب را بخوانیم؟ – روش مدرن یادگیری

این کتاب بر اساس ۴ اصل علم یادگیری طراحی شده:

1. Active Recall – به‌جای هایلایت، بپرس

بعد از هر بخش، کتاب از تو می‌پرسد – جواب را قبل از دیدن فکر کن.

چرا `typeof null === "object"` است؟ (جواب: باگ روز اول که برای سازگاری ماند)

2. Spaced Repetition – فصل ۳۷ واژه‌نامه

۱۴۰ اصطلاح را در ۷ روز مرور کن – هر روز ۲۰ تا. اپ Anki بساز.

♦ 3. Feynman Technique – به یک بچه توضیح بده

بعد از فصل Closure، سعی کن به یک دوست غیرفنی بگویی: "کلژر یعنی کوله‌پشتی که تابع با خود می‌برد."

♦ 4. Project-Based – فصل ۳۴

هر مفهوم را در ۸ مینی‌پروژه پیاده کن – از Debounce تا Kanban.

🚀 چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی فصل را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج آن را نام ببری؟
- ☐ می‌توانی آن را در پروژه واقعی استفاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه توضیح دهی؟

📅 مسیر صفر تا استخدام (پیشنهادی)

- ماه ۱: فصل ۱-۱۲ – روزی ۱ فصل + تمرین
- ماه ۲: فصل ۱۳-۲۴ – Event Loop و Proxy را با نقاشی بکش
- ماه ۳: فصل ۲۵-۳۲ – هر فصل یک PR با تست
- ماه ۴: فصل ۳۳-۳۸ – ۵۰ پرسش مصاحبه را ضبط کن و گوش بده

🔧 | ستاپ صفر – همین الان

```
BASH

# 1. Bun نصب
curl -fsSL https://bun.sh/install | bash

# 2. پروژه
mkdir js-deep && cd js-deep
bun init -y
bun add -d vitest biome

# 3. biome.json
echo '{"formatter":{"indentWidth":2},"linter":{"enabled":true}}' > biome.json

# 4. تست کن
echo "console.log('JS is love 🍷')" > index.js
bun index.js
```

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت Java و JavaScript؟ هیچ! نام JS فقط بازاریابی بود – مثل Car و Carpet.

Mid – چرا JS تک‌نخی است اما 1000 درخواست همزمان هندل می‌کند؟ چون I/O به libuv / Web API می‌رود؛ JS فقط Callback را در Queue می‌گذارد – Event Loop.

JIT – Senior چیست و چرا Deopt مهم است؟ JIT کامپایل در زمان اجراست. V8 فرض خوش‌بینانه می‌گیرد (مثلاً "این تابع همیشه number می‌گیرد")؛ اگر فرض بشکند، کد بهینه دور ریخته می‌شود (Deopt) و پرفورمنس سقوط می‌کند. درک این موضوع تفاوت 60fps و 10fps در انیمیشن است.

✓ جمع‌بندی و اتصال

- ♦ JS از یک اسکریپت ۱۰ روزه به پلتفرم جهان تبدیل شد.
 - ♦ V8 با Hidden Class و TurboFan آن را سریع کرد – و تو با کد Monomorphic به آن کمک می‌کنی.
 - ♦ اکوسیستم ۲۰۲۶: Bun + pnpm + Vite + Biome + Vitest + TS.
 - ♦ روش یادگیری: Active Recall + پروژه.
- فصل بعد:** محیط توسعه مدرن را می‌چینیم – از `type: module` تا `biome.json` – تا از روز اول حرفه‌ای باشی.

02 محیط توسعه مدرن – ستاپ حرفه‌ای ۲۰۲۶

ابزار درست، ۱۰ برابر بهره‌وری می‌دهد. این فصل را جدی بگیر – ستاپ بد، ۱۰۰ ساعت وقتت را می‌گیرد.

🎯 اهداف یادگیری

- ♦ تفاوت Node/Bun/Deno را با بنچمارک توضیح دهی
- ♦ پروژه ESM واقعی با `type: module` بسازی
- ♦ Biome را جایگزین ESLint+Prettier کنی
- ♦ VS Code را برای JS مدرن تنظیم کنی

🧠 مدل ذهنی: Runtime چیست؟

Runtime = موتور API + (V8) JS‌های اضافه (fs, fetch, etc) + Event Loop

- ♦ **Browser**: V8 + DOM + fetch + setTimeout
- ♦ **Node**: V8 + fs + http + libuv
- ♦ **Bun**: JavaScriptCore + fs + fetch + نیتیو TS transpiler داخلی

i چرا Bun سریع‌تر است؟

چون به جای V8 از JavaScriptCore (Safari) استفاده می‌کند و ماژول‌لودر را با Zig بازنویسی کرده – و TS را بدون `tsc` اجرا می‌کند.

گام ۱: Runtime

```
BASH

# Node 22 LTS (پایدار)
nvm install 22 && nvm use 22
node -v # v22.x

# Bun 1.2 (Dev سریع برای)
curl -fsSL https://bun.sh/install | bash
bun -v # 1.2.x

# تست سرعت
echo "console.log('hi')" > test.js
time node test.js
time bun test.js # 3x سریع تر
```

گام ۲: Package Manager – چرا pnpm؟

PM	node_modules	سرعت نصب	دیسک
npm	کپی	کند	زیاد
yarn	کپی	متوسط	زیاد
pnpm	symlink + content-addressable	سریع	70% کمتر

```
BASH

npm i -g pnpm
pnpm --version

mkdir my-app && cd my-app
pnpm init
pnpm add -D vitest biome @types/node
```

```
package.json

{
  "name": "js-deep",
  "type": "module",
  "scripts": {
    "dev": "bun --watch src/index.ts",
    "test": "vitest --coverage",
    "lint": "biome check --write ./src",
    "build": "vite build"
  }
}
```

```
src/index.ts – ESM واقعی

// ☒ ESM – استاتیک، Tree-shakable، Top-level await
import { readFile } from 'node:fs/promises';
import { sum } from './math.js';

export const config = await readFile('./config.json', 'utf8'); // Top-level await!

console.log(sum(1,2));

// ☐ CJS – bundler قدیمی، داینامیک، کند برای
// const fs = require('fs');
// module.exports = {}
```

⚠ هشدار – این کتاب فقط ESM

از این به بعد همه مثال‌ها ESM هستند. `require` را فراموش کن مگر مجبور به نگهداری Legacy باشی. دلیل: Tree Shaking، Code Splitting، و سازگاری مرورگر/سرور یکسان.

◆ گام ۴: Biome – یک ابزار به جای ۵ ابزار

biome.json – جایگزین ESLint + Prettier + import sorter

```
{
  "$schema": "https://biomejs.dev/schemas/1.9.0/schema.json",
  "formatter": {
    "enabled": true,
    "indentStyle": "space",
    "indentWidth": 2,
    "lineWidth": 100
  },
  "linter": {
    "enabled": true,
    "rules": {
      "recommended": true,
      "complexity": { "noUselessSwitchCase": "error" },
      "suspicious": { "noExplicitAny": "warn" }
    }
  },
  "javascript": {
    "formatter": { "quoteStyle": "single", "semicolons": "asNeeded" }
  }
}
```

BASH

```
pnpm biome check --write ./src # lint + format + sort imports 50,ms
```

◆ گام ۵: VS Code – تنظیمات Senior

.vscode/settings.json

```
{
  "editor.defaultFormatter": "biomejs.biome",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.biome": "explicit",
    "source.organizeImports.biome": "explicit"
  },
  "typescript.preferences.importModuleSpecifier": "relative",
  "emmet.includeLanguages": { "javascript": "javascriptreact" }
}
```

افزونه‌های ضروری:

Biome (biomejs.biome) ♦

Vitest (vitest.explorer) ♦

Error Lens ♦

GitLens ♦

عمیق: چگونه ESM Tree Shaking می‌کند؟ 🧐

utils.js

```
export const used = () => console.log("used");
export const unused = () => console.log("unused"); // اگر import حذف می‌شود!

// ✗ مانع Tree Shaking
export default { used, unused }; // object همه می‌ماند چون
```

ESM Bundler (Vite/Rollup) با تحلیل استاتیک `import/export` کد استفاده‌نشده را حذف می‌کند – فقط اگر ESM باشد و `sideEffects: false`.

تمرین فعال 📝

تمرین ۱: یک پروژه ESM بساز که `config.json` را با `Top-level await` بخواند و با `sum` جمع بزند.

تمرین ۲: یک فایل با ۱۰ خط کد بد (var, ==, no semicolon) بساز و با `biome check --write` درست کن – تفاوت را ببین.

چالش بازار کار: یک CLI کوچک با Bun بساز که یک فایل JS را می‌گیرد و تعداد `var` های آن را می‌شمارد – با `Bun.write` و `Bun.file`.

“سؤالات مصاحبه (با پاسخ)”

Mid – چرا ESM بهتر از CJS است؟ استاتیک، `Tree-shakable`، `Top-level await`، یکسان در مرورگر و سرور، و `import` می‌تواند lazy باشد (`import()`).

Senior – تفاوت Bun و Node در اجرای TS؟ Node نیاز به `--loader tsx` یا `tsc` دارد؛ Bun به‌صورت نیتیو TS/JSX را بدون transpile اجرا می‌کند و `node_modules` را با کش هوشمند می‌خواند – استارت‌آپ 3x سریع‌تر.

Senior – چرا pnpm از npm امن‌تر است؟ چون از symlink استفاده می‌کند و هر پکیج فقط به dependencyهای خودش دسترسی دارد، نه همه `node_modules` – جلوی Phantom Dependency را می‌گیرد.

چک لیست فهم ✓

□ می‌توانی توضیح دهی Runtime چیست و Bun چرا سریع‌تر است؟

□ می‌توانی پروژه ESM با Top-level await بسازی؟

□ Biome را جایگزین ESLint+Prettier کردی؟

□ VS Code را با formatOnSave تنظیم کردی؟

فصل بعد: وارد قلب JS می‌شویم — ۸ نوع داده، Value vs Reference، و Coercion که منشأ ۸۰٪ باگ‌هاست.

03 انواع داده و سیستم نوع – جایی که ۸۰٪ باگ‌ها متولد می‌شود

جاوااسکریپت فقط ۸ نوع داده دارد، اما همین ۸ نوع اگر با مدل حافظه فهمیده نشود، هر روز باگ می‌دهد. این فصل را ۲ بار بخوان.

🎯 اهداف یادگیری

- ♦ ۸ نوع داده را با `typeof` و مدل Stack/Heap توضیح دهی
- ♦ تفاوت Value vs Reference را روی کاغذ بکشی
- ♦ جدول Truthy/Falsy را حفظ باشی و `==` را برای همیشه کنار بگذاری
- ♦ `Object.is` و `===` را با مثال `NaN` و `-0` مقایسه کنی

🧠 مدل ذهنی: Stack vs Heap

تصور کن حافظه دو انبار است:

- ♦ **Stack (پشته):** کوچک، سریع، منظم – مثل کشوهای مرتب. Primitive‌ها اینجا هستند. LIFO.
- ♦ **Heap (توده):** بزرگ، کند، نامنظم – مثل انبار. Objects اینجا هستند. GC تمیز می‌کند.

stack-heap-visual.js

```
let a = 10;           // a در Stack: 10
let b = a;           // b در Stack: مستقل - 10 کپی
b = 20;
console.log(a); // 10 - چون کپی بود

let obj1 = { x: 10 }; // obj1 در Stack: 0 آدرس → {x:10} در Heap
let obj2 = obj1;     // obj2 در Stack: 0 آدرس - همان x123! مشترک!
obj2.x = 20;
console.log(obj1.x); // 20 - چون هر دو به یک خانه - 20 اشاره می‌کنند
```

Stack

Primitive + Reference (آدرس)
سریع، ثابت

Heap

Object واقعی
کند، پویا، GC



برای Object فقط آدرس را چک می‌کند!

i تشبیه دنیای واقعی

Primitive مثل فتوکپی برگه – هر کسی نسخه خودش را دارد. Object مثل لینک گوگل داک – همه یک سند را ادیت می کنند.

۸ نوع داده – عمیق

```
types-deep.js

typeof 42;           // "number" – IEEE 754 double
typeof 42n;          // "bigint" – برای اعداد بزرگ
typeof "hi";         // "string" – UTF-16
typeof true;         // "boolean"
typeof undefined;    // "undefined" – مقدار پیش فرض
typeof Symbol("id"); // "symbol" – کلید یکتا
typeof null;         // "object" – !باگ 1995 که ماند
typeof {};           // "object"
typeof (()=>{});      // "function" – شیء قابل صدا زدن

// چک امن null
value === null // ✓
typeof value === "object" && value !== null // می خواهی object اگر واقعاً
```

♦ باگ تاریخی typeof null

در نسخه اول JS، مقادارها با ۳ بیت type tag ذخیره می شدند: ۰۰۰ برای object، و null همه بیت ها ۰ بود – پس موتور فکر کرد object است! برای سازگاری هیچ وقت فیکس نشد.

Coercion – چالش‌های تبدیل ضمنی JS

JS سعی می‌کند خودش نوع را تبدیل کند – و همین باگ می‌سازد.

coercion-deep.js

```
"5" + 3 // "53" - + دهنده را ترجیح می‌دهد، رشته را به رشته تبدیل می‌کند
"5" - 3 // 2 - - تبدیل "5" را به 5 تبدیل می‌کند

[] + [] // "" - [].toString() → "" → "" + "" → ""
[] + {} // "[object Object]" - [].toString() + {}.toString()
{} + [] // 0 - {} به عنوان بلاک خالی 0 parse در کنسول کروم! چون {} به عنوان بلاک خالی 0 می‌شود، بعد +[] → "" + → 0

Boolean([]) // true - truthy مقدار 8 فقط است! falsy داریم
Boolean("") // false
```

مقدار	Boolean	توضیح
false, 0, -0, 0n, "", null, undefined, NaN	falsy	فقط این ۸ تا
بقیه - حتی [], {}, "0"	truthy	همه truthy

✗ اشتباه مرگبار – در Code Review رد می‌شوی

JS

```
// ✗
if (user.age == 0) { /* "0" == 0 → true - رشته‌ای هم قبول می‌شود */ }

// ✓
if (user.age === 0) { /* فقط عدد 0 */ }
if (Number(user.age) === 0) { /* اگر می‌خواهی تبدیل کنی، صریح */ }
```

قانون طلایی این کتاب: همیشه === و !== - هرگز == و !=

equality-deep.js

```
// 1. === – Strict Equality (بدون Coercion)
NaN === NaN // false
-0 === +0    // true

// 2. Object.is – دقیق‌ترین (ES6)
Object.is(NaN, NaN) // true – درست!
Object.is(-0, +0)   // false – فرق می‌گذارد!
Object.is(NaN, NaN) // true

// 3. == – Abstract Equality (با Coercion) – هرگز استفاده نکن!
// [] == ![] → true – چرا؟!
// ![] → false
// [] == false → [].toString() == false → "" == false → 0 == 0 → true
```

♦ مقایسه عمیق اشیا

deep-equal.js

```
// ✗
{ a:1 } === { a:1 } // false – آدرس متفاوت

// ✔ روش‌های درست
JSON.stringify({a:1}) === JSON.stringify({a:1}) // کار می‌کند ولی Date/Function را خراب می‌کند
structuredClone // برای کپی عمیق
یا
import { isEqual } from 'lodash-es';
isEqual({a:1}, {a:1}) // true – عمیق

// بهترین: مقایسه با id
users.find(u => u.id === other.id) // به جای مقایسه کل شیء
```

تمرین فعال – Active Recall

تمرین ۱ – پیش‌بینی کن، بعد اجرا کن:

```
JS

console.log([] + []); // ?
console.log([] + {}); // ?
console.log({} + []); // ? (!در کنسول و در فایل فرق دارد)
console.log(Boolean([])); // ?
console.log(Boolean("0")); // ?
```

تمرین ۲ – باگ پیدا کن:

```
JS

function isEmpty(arr) {
  if (arr == false) return true; // می‌شود true هم arr=[0] ولی arr=[] → true باگ کجاست؟
}
// درست:
function isEmptySafe(arr) {
  return Array.isArray(arr) && arr.length === 0;
}
```

چالش بازار کار: تابعی بنویس `deepClone` که `structuredClone` را شبیه‌سازی کند – با `Map` برای `circular` `reference`.

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت null و undefined؟ undefined یعنی متغیر تعریف شده ولی مقدار نگرفته (پیش فرض JS)؛ null یعنی تو عمداً گفتی "هیچ" – قرارداد تیمی.

Mid – چرا `[] == []` true است؟ زنجیره Coercion: `!![] → false → [] == false → " " == false → true`

Senior – چرا `Object.is` بهتر از `===` برای NaN است؟ چون `===` برای NaN false می دهد (باگ IEEE 754) ولی `Object.is(NaN, NaN)` true – و برای `-0` vs `+0` هم دقیق تر است. در React و Redux برای مقایسه state از `Object.is` استفاده می شود.

Senior – تفاوت Value vs Reference در پارامتر تابع؟

```
JS
function foo(obj) { obj.x = 10; obj = {x:20}; }
let o = {x:1}; foo(o); console.log(o.x) // 10
// چون obj روی Heap آدرس است، تغییر ویژگی روی obj فقط می کند Stack فقط کپی reassign اثر دارد ولی Heap
```

✓ چک لیست فهم عمیق

- می توانی Stack vs Heap را روی کاغذ بکشی و توضیح دهی چرا `obj2=obj1` مشترک است؟ ☐
- جدول ۸ falsy را حفظی؟ ☐
- می توانی `[] + {}` را مرحله به مرحله Coercion کنی؟ ☐
- می توانی توضیح دهی چرا همیشه `===`؟ ☐

فصل بعد: وارد Scope می شویم – جایی که `var` می میرد و `let` با TDZ متولد می شود.

04 متغیرها، Scope و Hoisting – از صفر تا TDZ و Lexical

جایی که متغیر را تعریف می‌کنی، سرنوشت باگ، امنیت و پرفورمنس کدت را تعیین می‌کند. این فصل را اگر عمیق بفهمی، ۵۰٪ باگ‌های JS برای همیشه حل می‌شود.

🎯 اهداف یادگیری

- ♦ تفاوت var/let/const را با مدل حافظه و Hoisting توضیح دهی
- ♦ TDZ را نقاشی کنی و بگویی چرا `typeof` در TDZ خطا می‌دهد
- ♦ Lexical Scope را با مثال closure توضیح دهی
- ♦ ۳ قانون طلایی تعریف متغیر برای Production بگویی

🤔 چرا Scope مهم‌ترین مفهوم بعد از Type است؟

در مصاحبه Senior همیشه می‌پرسند:

"چرا `for(var i...) setTimeout(3,3,3)` چاپ می‌کند ولی با `let` درست است؟"

جواب فقط Scope است. Scope تعیین می‌کند متغیر کجا زنده است، کجا می‌میرد، و چه کسی به آن دسترسی دارد – پایه Closure، Module و Private.

مدل ذهنی: Scope = جعبه‌های تودرتو 🧠

تصور کن کدت جعبه‌های تودرتو است – هر `{ }` یک جعبه جدید.

♦ **Global Scope**: بزرگ‌ترین جعبه – همه می‌بینند – خطرناک

♦ **Function Scope**: جعبه تابع – `var` اینجا زندگی می‌کند

♦ **Block Scope**: جعبه `{ }` – `let/const` اینجا – امن

```
scope-visual.js

let global = "من همه‌جا هستم";

function outer() {
  let outerVar = "outer من در";
  {
    let blockVar = "من فقط در این { }";
    console.log(blockVar); // ✓ می‌بیند
    console.log(outerVar); // ✓ جعبه والد را می‌بیند
    console.log(global); // ✓ Global را می‌بیند
  }
  console.log(blockVar); // ✗ ReferenceError – جعبه فرزند را نمی‌بیند!
}
```

Global

window, globalThis
همه می‌بینند – خطر pollution



Function

var اینجا
قدیمی، function-scope



Block

let/const اینجا
مدرن، امن، قابل پیش‌بینی

چرا var منسوخ شد؟ – ۳ ایراد اساسی

♦ ایراد اساسی ۱: Hoisting با undefined

```
var-hoist.js

console.log(x); // undefined – چرا؟ چون:
var x = 10;

// در واقع:
var x; // Hoist به بالا – مقدار undefined
console.log(x); // undefined
x = 10;
```

ایراد اساسی ۲: Function-Scope نه Block-Scope

```
var-block.js

if (true) {
  var y = 10;
}
console.log(y); // بیرون آمد if از 10 -
// با let:
if (true) {
  let z = 10;
}
console.log(z); // ReferenceError - امن ✓
```

ایراد اساسی ۳: باگ معروف حلقه

```
loop-bug.js

for (var i=0; i<3; i++) {
  setTimeout(()=> console.log(i), 100);
}
// 3,3,3 - چرا؟! 0,1,2 نه
// اشاره می‌کنند که در پایان 3 شده i به همان callback دارد و هر سه i یک var چون

// ✓ با let - binding جدا (per-iteration)
for (let i=0; i<3; i++) {
  setTimeout(()=> console.log(i), 100); // 0,1,2
}

// ✓ قدیمی با IIFE
for (var i=0; i<3; i++) {
  ((j)=> setTimeout(()=> console.log(j),100))(i);
}
```

✗ در Code Review رد می‌شوی اگر var بنویسی

از 2015 به بعد، هیچ دلیل موجهی برای var نیست. ESLint قانون no-var دارد - فعال کن.

⚡ TDZ – ناحیه مرده زمانی

Temporal Dead Zone – فاصله بین شروع بلاک و خط تعریف `let/const`. دسترسی در TDZ خطا می‌دهد – عمداً! برای گرفتن باگ.

```
tdz-deep.js

{
  // شروع - از اینجا تا خط تعریف a برای TDZ
  // console.log(a); // ReferenceError: Cannot access 'a' before initialization
  // typeof a; // TDZ! هم خطا می‌دهد در typeof حتی (var برخلاف)

  let a = 10; // تمام TDZ
  console.log(a); // 10
}

// خوب است؟ چون باگ را زود می‌گیرد TDZ چرا
// ساکت می‌ماند و باگ پنهان می‌ماند var: undefined با
// fail-fast - فوراً خطا با let
```

🧠 Scope – Lexical Scope در زمان نوشتن، نه اجرا

Scope در زمان نوشتن کد (lexical) تعیین می‌شود، نه زمان اجرا (dynamic) – برخلاف `this`.

```
lexical-deep.js

const name = "global";

function outer() {
  const name = "outer";
  function inner() {
    console.log(name); // "outer" - lexical نزدیک‌ترین جعبه
  }
  return inner;
}

const fn = outer();
fn(); // "outer" - نه "global"! چون inner در outer نوشته شده، نه در outer
// مثل آدرس خانه در شناسنامه - هر جا بری، شناسنامه‌ات با توست Scope: تشبیه
// (فصل بعد) Closure محل تولدش را با خود می‌برد - این می‌شود Scope، تابع هر جا صدا زده شود
```


const به معنی Immutable نیست!

const-deep.js

```
const user = { name: "Ali" };
user.name = "Reza"; // ☒ مجاز - مرجع ثابت، محتوا نه
// user = {} // ☐ خطا - مرجع عوض نمی‌شود

const arr = [1,2];
arr.push(3); // ☒ مجاز - محتوا عوض می‌شود

// واقعی immutability برای:
Object.freeze(user); // نمی‌کند freeze سطحی - تودرتو را
// یا
import { produce } from "immer";
const next = produce(user, draft => { draft.name = "Reza"; }); // واقعی immutable
```

✦ قانون طلایی Seniorها

همیشه پیش‌فرض `const`، اگر نیاز به `reassign` بود `let`، هرگز `var` ننویس. این قانون ساده 90٪
باگ‌های `scope` را حذف می‌کند و کدت را قابل پیش‌بینی می‌کند.

تمرین فعال

تمرین ۱ - پیش‌بینی:

JS

```
console.log(a); var a=1;
console.log(b); let b=2;
```

تمرین ۲ - باگ حلقه را با ۳ روش حل کن: `let`، `ba IIFE`، `ba forEach`.

چالش بازار کار: یک ماژول `counter.js` بساز که `count` خصوصی دارد و فقط `inc()` و `get()` `export` می‌کند -
بدون کلاس، فقط با `Closure` و `let`.

“ سؤالات مصاحبه (با پاسخ)

Junior – hoisting چیست؟ بالا کشیده شدن تعریف `var` و `function declaration` به ابتدای `scope`. `let/const` هم `hoist` می‌شوند ولی در `TDZ` می‌مانند و دسترسی خطا می‌دهد.

Mid – چرا `typeof` در `TDZ` خطا می‌دهد؟ چون `TDZ` برای گرفتن باگ طراحی شده – حتی `typeof` هم نباید به متغیر نرسیده دسترسی دهد.

Senior – چرا `IIFE` در `ES5` لازم بود؟ چون `var block-scope` نداشت؛ برای ساخت `scope` خصوصی از `(function(){})( )` استفاده می‌کردند. الان با `block {}` و `let` نیازی نیست – `IIFE` منسوخ.

Senior – تفاوت `Scope` و `Context (this)`؟ `Scope` در زمان نوشتن تعیین می‌شود (`lexical`) و متغیرها را پیدا می‌کند؛ `Context (this)` در زمان اجرا و بر اساس نحوه فراخوانی تعیین می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی ۳ ایراد اساسی `var` را با مثال بگویی؟
- ☐ می‌توانی `TDZ` را نقاشی کنی؟
- ☐ می‌توانی باگ حلقه `var` را با `let` حل کنی و توضیح دهی چرا `let per-iteration binding` می‌سازد؟
- ☐ می‌توانی توضیح دهی چرا `const` به معنی `immutable` نیست؟

فصل بعد: وارد قلب `JS` می‌شویم – تابع – که هم مقدار است، هم شیء، هم کلژر، هم کارخانه.

05 عملگرها و کنترل جریان – از if تا الگوهای مدرن ۲۰۲۵

کد تمیز، کد تخت است – بدون تو در تو عمیق. این فصل یاد می‌گیری چطور با عملگرهای مدرن، کدت را امن و خوانا کنی.

🎯 اهداف یادگیری

- ♦ تفاوت `??` و `||` را با مثال `0` توضیح دهی
- ♦ `?.` را برای ۳ سطح تودرتو استفاده کنی
- ♦ کد تو در تو را با Early Return تخت کنی
- ♦ Destructuring با default و rename را حرفه‌ای استفاده کنی

🧠 مدل ذهنی: کد تخت = کد قابل فهم

مغز انسان 2±۷ چیز را همزمان نگه می‌دارد. کد تو در تو با ۴ if، مغز را خسته می‌کند.

```
JS

// ✗ سخت - تو در تو
function process(user) {
  if (user) {
    if (user.active) {
      if (user.age >= 18) {
        return doWork(user);
      }
    }
  }
}

// ✓ تخت - Early Return - آسان
function processFlat(user) {
  if (!user) return null;
  if (!user.active) return "inactive";
  if (user.age < 18) return "underage";
  return doWork(user); // بدون تو در تو
}
```

Optional Chaining (?.) – پایان Cannot read property

```
optional.js

const city = user?.address?.city ?? "نامشخص";
const len = arr?.[0]?.name?.length; // حتی برای آرایه و تابع
const result = fn?.(); // تابع بود، صدا بزن fn اگر

// قدیمی:
// const city = user && user.address && user.address.city;
// جدید:
const cityNew = user?.address?.city; // کوتاه، امن، خوانا
```

Nullish Coalescing (??) – فقط null/undefined

```
nullish.js

const count = inputCount ?? 10; // اگر inputCount=0، درست - همان 0 می ماند
const count2 = inputCount || 10; // ❌ 0 باگ - 10 می کند!

// جدول:
0 ?? 10 // 0 - 0 نیست null/undefined چون
0 || 10 // 10 - 0 است falsy چون
"" ?? "default" // "" - "" رشته خالی می ماند
"" || "default" // "default" - falsy

// قانون: || برای مقدار پیش فرض عدد/رشته، همیشه ?? - نه
```

Logical Assignment – ES2021

```
logical-assign.js

let a = null;
a ??= 10; // a = a ?? 10 → 10
a ||= 20; // a = a || 20 → 10 چون truthy می ماند
a &&= 30; // اگر truthy 30 بود،

// کاربرد واقعی:
user.settings ??= {}; // اگر settings بساز نداشت،
```

separator.js

```
const price = 1_000_000_000; // خواناتر از 10000000000
const mask = 0b1111_0000_1010;
```

const {name} = user — فراتر از Destructuring 📦

destructure-pro.js

```
const user = { id:1, name:"Ali", address:{ city:"Tehran" }, scores:[90,85,95], role:undefined };
```

```
// Rename + Default + Nested + Rest
```

```
const {
  name: userName, // rename
  age = 18, // default
  address: { city }, // nested
  scores: [first, ...rest], // array rest
  role = "user" // default اگر undefined
} = user;
```

```
// Swap بدون متغیر موقت
```

```
let x=1, y=2;
[x,y] = [y,x]; // جادو!
```

```
// پارامتر نام‌دار - بهترین برای تابع با ۳+ پارامتر
```

```
function createUser({ name, email, role="user", verified=false }={}) {
  return { name, email, role, verified };
}
```

```
createUser({ name:"Ali", email:"ali@test.com" }); // خوانا - ترتیب مهم نیست
```

کنترل جریان – از if تا Pattern Matching

control-pro.js

```
// زنجیره‌ای if-else به جای:
const getRoleText = (role) => ({
  admin: "مدیر کل",
  editor: "ویراستار",
  user: "کاربر عادی"
}[role] ?? "ناشناس");

// طولانی switch به جای:
const handlers = {
  click: () => console.log("click"),
  hover: () => console.log("hover"),
  scroll: () => console.log("scroll")
};
const handle = (type) => (handlers[type] ?? (() => console.log("unknown"))());

// for...of vs for...in
for (const key in obj) { /* ✗ کلیدها + prototype – برای object */ }
for (const val of arr) { /* ✓ مقادیر – برای آرایه */ }
for (const [k,v] of Object.entries(obj)) { /* ✓ کلید+مقدار */ }
```

⚠ هشدار – for...in برای آرایه ممنوع

for...in ترتیب تضمینی ندارد و کلیدهای prototype را هم می‌آورد. همیشه for...of یا forEach برای آرایه.

تمرین فعال

تمرین ۱: یک تابع `getUserCity(user)` بنویس که با `?.` و `??` شهر را برگرداند یا "نامشخص" – بدون `if`.

تمرین ۲: کد زیر را با Early Return تخت کن:

```
function save(user) {
  if (user) {
    if (user.valid) {
      if (user.age >= 18) {
        db.save(user);
      }
    }
  }
}
```

چالش: یک تابع `formatPrice(price)` بنویس که با `Intl.NumberFormat("fa-IR")` قیمت را فارسی کند و اگر `price` null بود، "رایگان" برگرداند – با `??`.

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت `??` و `||` ؟ `||` هر `falsy` (0, "", false) را جایگزین می‌کند؛ `??` فقط `null/undefined` – برای مقدار پیش‌فرض عددی، `??` امن‌تر.

Senior – چرا `obj?.a?.b` از `obj.a && obj.a.b` بهتر است؟ کوتاه‌تر، امن‌تر، short-circuit دقیق‌تر، و برای تابع `fn?.()` و آرایه `arr?.[0]` هم کار می‌کند – در حالی که `&&` برای تابع و آرایه خطرناک است.

✓ چک لیست

- ☐ تفاوت `??` و `||` را با مثال 0 می‌فهمی؟
- ☐ می‌توانی کد تو در تو را با Early Return تخت کنی؟
- ☐ Destructuring با `default` و `rename` را حرفه‌ای استفاده می‌کنی؟

فصل بعد: قلب JS – تابع – که ۴ چهره دارد.

06 توابع – قلب جاوااسکریپت – ۴ چهره یک مفهوم

اگر به تو بگویند فقط یک فصل از JS را عمیق بخوان، این فصل را بخوان. تابع در JS فقط تابع نیست؛ مقدار است، شیء است، کلژ است، کارخانه است. اگر تابع را بفهمی، JS را فهمیده‌ای.

🎯 اهداف یادگیری

- ۴ روش تعریف تابع و تفاوت Hoisting آن‌ها را بگویی
- ۴ چیزی که Arrow Function ندارد را حفظ باشی
- ۴ First-Class Function را با ۳ مثال (callback, return, property) توضیح دهی
- this در Arrow vs Function را با مثال setTimeout بگویی

🧠 مدل ذهنی: تابع = ماشین + کوله‌پشتی + کارخانه

- ماشین: ورودی → خروجی (Pure)
- کوله‌پشتی: Closure – متغیرهای بیرون را با خود می‌برد
- کارخانه: با new شیء می‌سازد
- شیء: خودش ویژگی دارد (fn.value = 0)

📦 ۴ روش تعریف – عمیق

func-deep.js

```
// 1. Declaration – Hoist می‌شود – قابل صدا زدن قبل از تعریف
console.log(add(1,2)); // کار می‌کند – 3
function add(a,b) { return a+b; }

// 2. Expression – Hoist نمی‌شود – باید بعد از تعریف صدا بزنی
// console.log(add2(1,2)); // ReferenceError
const add2 = function(a,b) { return a+b; };

// 3. Arrow – this, arguments, prototype, super ندارد – سبک‌تر
const add3 = (a,b) => a+b;
const add3Block = (a,b) => { return a+b; }; // {} نیاز به return

// 4. Function Constructor – eval مخفی – هرگز!
const add4 = new Function('a','b','return a+b'); // امنیت + پرفورمنس فاجعه ✗
```


♦ چرا ۴ Arrow Function چیز ندارد؟

چون سبک‌تر و برای callback ساخته شده – نه برای متد و سازنده.

```
arrow-limits.js

const user = {
  name: "Ali",
  // ✗ Arrow متد – به‌عنوان متد – this ندارد!
  greetArrow: () => console.log(this.name), // this = window – نه user!

  // ✓ Function متد – به‌عنوان متد – this دارد
  greet() { console.log(this.name); }, // this = user

  // ✓ Arrow متد – داخل متد – this را از متد می‌گیرد – عالی برای
  greetTimeout() {
    setTimeout(() => {
      console.log(this.name); // "Ali" – this از greetTimeout
    }, 100);
  }
};
```

First-Class – تابع به عنوان مقدار 📁

در JS تابع یک شهروند درجه یک است – مثل عدد و رشته:

```
first-class-deep.js

// 1. - callback - می‌تواند آرگومان باشد
[1,2,3].map(x => x*2);
setTimeout(()=> console.log("hi"), 100);

// 2. - factory, closure - می‌تواند برگردانده شود
const makeAdder = (n) => (x) => x+n; // تابع می‌سازد
const add5 = makeAdder(5);
add5(10); // 15 - add5 کوله‌پشتی n=5 را دارد

// 3. - می‌تواند ویژگی داشته باشد - شیء است
function counter() { return counter.value++; }
counter.value = 0;
counter(); // 0
counter(); // 1

// 4. - می‌تواند در آرایه و شیء باشد
const ops = {
  add: (a,b)=> a+b,
  mul: (a,b)=> a*b
};
ops.add(2,3); // 5
```

پارامترها – Rest, Default, Destructuring 🔧

params-deep.js

```
function sum(a, b=0, ...rest) { // default + rest
  console.log(a, b, rest);
  return a + b + rest.reduce((s,x)=>s+x,0);
}
sum(1); // 1,0,[]
sum(1,2,3,4); // 1,2,[3,4] → 10

// Spread – باز کردن
const arr = [1,2];
sum(...arr, 3,4); // 1,2,3,4

// پارامتر نام‌دار – بهترین برای 3+ پارامتر
function createUser({ name, email, role="user", verified=false }={}) {
  return { name, email, role, verified };
}
createUser({ name:"Ali", email:"a@test.com" }); // خوانا – ترتیب مهم نیست
```

تمرین فعال 🧪

تمرین ۱: تابعی بنویس `once(fn)` که `fn` را فقط یک‌بار اجرا کند – با Closure.

تمرین ۲: باگ زیر را حل کن:

JS

```
const obj = { value:10, get: () => this.value };
obj.get(); // undefined – چرا؟ درست کن
```

چالش بازار کار: یک `memo` بساز که نتیجه تابع را کش کند – با Map و Closure – مثل فصل ۷.

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت declaration و expression؟ declaration hoist می‌شود و قبل از تعریف قابل صدا زدن؛ expression نه – باید بعد از تعریف.

Mid – چرا Arrow Function برای متد کلاس مناسب نیست؟ چون هر نمونه یک کپی جدا از Arrow می‌سازد (روی prototype نیست) و حافظه بیشتر؛ به علاوه `this` ثابت و `super` ندارد.

Senior – چرا `new Function` خطرناک است؟ چون مثل `eval` رشته را به عنوان کد اجرا می‌کند – XSS و عدم بهینه‌سازی V8.

Senior – تفاوت `function` و `Arrow` در `arguments`؟ `function` `arguments` شبه‌آرایه دارد؛ `Arrow` ندارد – باید از `...rest` استفاده کنی.

✓ چک‌لیست

☐ ۴ روش تعریف و Hoisting آن‌ها را می‌فهمی؟

☐ ۴ چیز که `Arrow` ندارد را حفظی؟

☐ `First-Class` را با ۳ مثال می‌گویی؟

☐ باگ `this` در `setTimeout` را با `Arrow` حل می‌کنی؟

فصل بعد: Closure – کوله‌پشتی که تابع با خود می‌برد – مهم‌ترین مفهوم JS.

07 کلژر و زنجیره Scope – مهم‌ترین مفهوم JS

اگر کسی به تو بگوید JS را در یک کلمه بگو، بگو Closure. کلژر یعنی تابع + محیطی که در آن متولد شده – و این محیط حتی بعد از مرگ والد، زنده می‌ماند.

🎯 اهداف یادگیری

- ◆ کلژر را با نقاشی Stack/Heap توضیح دهی
- ◆ ۴ کاربرد واقعی: ماژول خصوصی، Memo، Once، Factory
- ◆ باگ حلقه var را با ۳ روش حل کنی و بگویی چرا let درست است
- ◆ توضیح دهی چرا کلژر می‌تواند Memory Leak بدهد

🧠 مدل ذهنی: تابع = کد + کوله‌پشتی

هر تابع هنگام تولد، یک کوله‌پشتی از متغیرهای بیرونش برمی‌دارد – و هر جا برود، کوله‌پشتی با اوست.

```
closure-backpack.js

function makeCounter() {
  let count = 0; // می‌ماند – چون کسی به آن اشاره دارد Heap در
  return function() { // این تابع کوله‌پشتی را دارد
    return ++count;
  };
}

const c1 = makeCounter(); // count=0 + جعبه جدید
c1(); // 1 – کوله‌پشتی c1
c1(); // 2 – همان کوله‌پشتی
const c2 = makeCounter(); // count=0 + جعبه جدید جدا
c2(); // 1 – کوله‌پشتی c2 – مستقل!
```

Lexical Env

متغیرها + مرجع به Parent
هر {} یک Env

Closure

تابع + زنجیره Env
کوله‌پشتی

Heap

تا تابع زنده است، Env هم زنده است
GC نمی‌شود

i تشبیه دنیای واقعی

تابع مثل بچه‌ای است که از خانه والد بیرون می‌رود ولی کلید خانه والد را در جیب دارد – هر وقت خواست برمی‌گردد و از یخچال والد برمی‌دارد! این کلید، Closure است.

۴ کاربرد واقعی – که در بازار کار هر روز استفاده می‌شود

۱. ماژول خصوصی – بدون کلاس

```
private-module.js

const Bank = (() => {
  let balance = 0; // خصوصی - بیرون دسترسی نیست!
  return {
    deposit: (n) => balance += n,
    withdraw: (n) => balance -= n,
    get: () => balance
  };
})();
Bank.deposit(100);
Bank.balance // undefined - خصوصی! ✓
Bank.get() // 100
```

۲. Factory – کارخانه تابع‌ساز

```
factory.js

const makeMultiplier = (x) => (y) => x*y;
const double = makeMultiplier(2); // کوله‌پشتی x=2
const triple = makeMultiplier(3); // کوله‌پشتی x=3
double(5); // 10
triple(5); // 15

// کاربرد prefix: لاگر با
const makeLogger = (prefix) => (msg) => console.log(`[${prefix}] ${msg}`);
const errorLog = makeLogger("ERROR");
errorLog("fail"); // [ERROR] fail
```

۳. Memoization – کش هوشمند

memo-deep.js

```
function memo(fn) {
  const cache = new Map(); // کوله‌پشتی cache
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      console.log("from cache");
      return cache.get(key);
    }
    const res = fn(...args);
    cache.set(key, res);
    return res;
  };
}

const fib = memo((n) => n <= 1 ? n : fib(n-1)+fib(n-2));
fib(40); // بار اول کند
fib(40); // cache بار دوم فوری – از
```

۴. Once – فقط یک‌بار

once.js

```
const once = (fn) => {
  let done = false, result;
  return (...args) => {
    if (!done) { result = fn(...args); done = true; }
    return result;
  };
};

const init = once(() => console.log("init once"));
init(); // init once
init(); // هیچ – چون done=true
```

باگ معروف حلقه – عمیق

loop-deep.js

```
// ✗ با var – مشترک i یک
for (var i=0; i<3; i++) {
  setTimeout(()=> console.log(i), 100); // 3,3,3
}
// اشاره می‌کنند که در پایان 3 شده i به همان callback دارد – هر سه i یک var چرا؟

// ✓ با let – binding جدا (per-iteration) – ES6
for (let i=0; i<3; i++) {
  setTimeout(()=> console.log(i), 100); // 0,1,2 – جدا جعبه جدا
}

// ✓ با IIFE – قدیمی
for (var i=0; i<3; i++) {
  ((j)=> setTimeout(()=> console.log(j),100))(i); // i کی j
}
```

حافظه – کلژر می‌تواند Leak بدهد

JS

```
function leak() {
  const big = new Array(1e6).fill("*"); // 1M
  return () => console.log(big[0]); // big هیچ‌وقت GC نمی‌شود چون closure داشته
}
// راه‌حل: فقط چیزی که نیاز داری نگه دار
function noLeak() {
  const big = new Array(1e6).fill("*");
  const first = big[0]; // فقط first
  return () => console.log(first); // big آزاد می‌شود
}
```

در DevTools → Memory → Heap Snapshot بگیر – اگر closure بزرگ می‌بینی، leak داری.

تمرین فعال

تمرین ۱: یک `makeCounter` بساز که `inc()`، `dec()`، `get()` داشته باشد – با `closure`.

تمرین ۲: باگ حلقه را با `let` حل کن و با نقاشی توضیح بده چرا `let per-iteration` است.

چالش بازار کار: یک `debounce` با `closure` بساز – تایمر را در `closure` نگه دار.

“ سؤالات مصاحبه (با پاسخ)

Mid – کلژر چیست؟ تابعی که به متغیرهای scope بیرونی حتی بعد از اتمام آن scope دسترسی دارد – چون Env در Heap می‌ماند.

Senior – چرا کلژر در لوپ با var مشکل دارد؟ چون var یک binding دارد و هر سه callback به همان i اشاره می‌کنند که 3 شده؛ let برای هر تکرار binding جدا می‌سازد (spec ES6).

Senior – فرق Closure و Scope Chain؟ Scope Chain زنجیره جستجوی متغیر در زمان اجراست؛ Closure نتیجه نگه داشتن آن زنجیره توسط تابع بعد از خروج از scope است – مثل کلید خانه والد.

Senior – چرا React Hooks کلژر است؟ چون `useState` مقدار state را در closure نگه می‌دارد و `useEffect` callback آن closure را می‌بیند – به همین دلیل dependency array لازم است.

✓ چک لیست

☐ می‌توانی Closure را با کوله‌پشتی توضیح دهی؟

☐ ۴ کاربرد واقعی را با کد می‌نویسی؟

☐ باگ حلقه var را با ۳ روش حل می‌کنی؟

☐ می‌توانی Memory Leak با closure را تشخیص دهی؟

فصل بعد: Prototype – زنجیری که همه اشیا به آن وصل‌اند.

08 اشیا و پروتوتایپ – راز ارث‌بری JS

در JS کلاس وجود ندارد – فقط شیء و لینک به شیء دیگر. اگر Prototype را بفهمی، می‌فهمی چرا `{}.toString` کار می‌کند در حالی که تو تعریفش نکرده‌ای.

🎯 اهداف یادگیری

- ◆ Prototype Chain را از `{}` تا `null` نقاشی کنی
- ◆ تفاوت `__proto__` و `prototype` را با مثال `new` توضیح دهی
- ◆ کلاس را به Prototype تبدیل کنی – بفهمی قند است
- ◆ `Object.create(null)` و Composition را به‌جای ارث‌بری عمیق استفاده کنی

🧠 مدل ذهنی: هر شیء یک لینک مخفی دارد

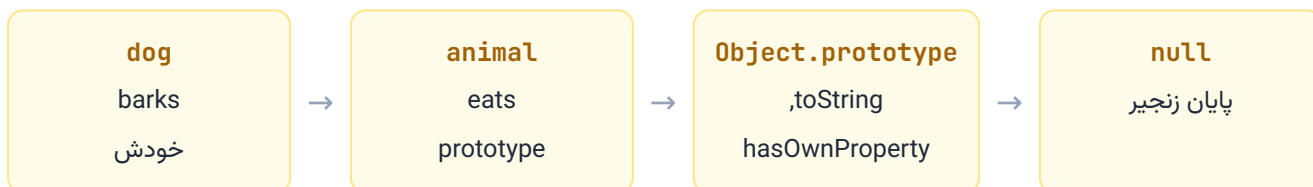
هر شیء یک لینک پنهان `[[Prototype]]` دارد – مثل زنجیر.

proto-visual.js

```
const animal = { eats: true };
const dog = { barks: true };
Object.setPrototypeOf(dog, animal); // dog.__proto__ = animal

dog.barks // true – خودش دارد
dog.eats // true – می‌خواند prototype ندارد، از
dog.toString // از Object.prototype

// (پایان) dog → animal → Object.prototype → null زنجیره
```



i تشبیه

Prototype مثل دفترچه راهنمای خانوادگی – اگر خودت بلد نیستی، از پدر می‌پرسی، اگر پدر هم بلد نیست از پدربزرگ – تا `null` که می‌گوید "نمی‌دانم".

Constructor و ۴ – new کار مخفی 🔑

new-deep.js

```
function User(name) {
  this.name = name; // this = شیء جدید
}
User.prototype.greet = function() { return `Hi ${this.name}`; };

const u = new User("Ali");
u.greet(); // Hi Ali

// new چه می‌کند؟ (مهم برای مصاحبه)
function myNew(Constructor, ...args) {
  const obj = {}; // 1. {} می‌سازد
  Object.setPrototypeOf(obj, Constructor.prototype); // 2. __proto__ را وصل می‌کند
  const result = Constructor.apply(obj, args); // 3. this را bind می‌کند
  return result instanceof Object ? result : obj; // 4. اگر return چیزی نبود، this را برمی‌گرداند
}
```

کلاس – فقط Syntax Sugar 🧐

class-sugar.js

```
class User {
  #password; // private field – ES2022 – واقعاً خصوصی!
  constructor(name, password) { this.name = name; this.#password = password; }
  greet() { return `Hi ${this.name}`; }
  static createAnon() { return new User("Anonymous", "123"); }
}

// است prototype زیر کاپوت همان:
typeof User // "function" – کلاس تابع است!
User.prototype.greet // function – است prototype متد روی
Object.getPrototypeOf(new User("Ali")) === User.prototype // true

// Private field _field است واقعاً خصوصی – برخلاف
const u = new User("Ali", "secret");
u.#password // SyntaxError – بیرون دسترسی نیست!
u._password // اگر _ بود، دسترسی داشت – فقط قرارداد
```

composition-deep.js

```
// ✗ ارث‌بری عمیق - شکننده - تغییر والد همه را می‌شکند
class Animal {}
class Bird extends Animal {}
class FlyingBird extends Bird {}
class Parrot extends FlyingBird {} // سطح 4!

// ✓ Composition - انعطاف‌پذیر - مثل لگو
const canEat = { eat() { console.log("eating"); } };
const canWalk = { walk() { console.log("walking"); } };
const canFly = { fly() { console.log("flying"); } };

const person = { ...canEat, ...canWalk, name: "Ali" };
person.eat(); person.walk();

const bird = { ...canEat, ...canFly, name: "Parrot" };
bird.eat(); bird.fly();

// کلاس Mixin یا با
const Flyable = Base => class extends Base { fly() { console.log("flying"); } };
const Eatable = Base => class extends Base { eat() { console.log("eating"); } };
class Parrot extends Flyable(Eatable(Object)) {}
```

✦ قانون طلایی

Composition over Inheritance – تا جایی که می‌توانی از ترکیب استفاده کن، نه ارث‌بری عمیق. ارث‌بری فقط برای "is-a" واقعی – مثل `Admin is a User`.

✦ تمرین

تمرین ۱: زنجیره `[]` را نقاشی کن: `[] → Array.prototype → Object.prototype → null` – چرا `{}.map` کار می‌کند؟

تمرین ۲: یک `myNew` بنویس که `new` را شبیه‌سازی کند.

چالش: با `Object.create(null)` یک دیکشنری امن بساز که `__proto__` pollution نشود.

“ سؤالات مصاحبه (با پاسخ)

Junior – prototype چیست؟ شیئی که به عنوان الگو برای اشیای دیگر استفاده می شود؛ اگر ویژگی در خود شیء نبود، از prototype می خواند.

Mid – تفاوت __proto__ و prototype ؟ ویژگی تابع سازنده است که برای اشیای ساخته شده با new به عنوان __proto__ قرار می گیرد؛ __proto__ ویژگی خود شیء است که به والد اشاره می کند – منسوخ، از Object.getPrototypeOf استفاده کن.

Senior – چرا کلاس ها در JS ارث بری کلاسیک نیستند؟ چون ارث بری JS delegation است نه کپی؛ this همیشه به شیء فراخوانی کننده اشاره می کند، نه جایی که متد تعریف شده – برخلاف Java که کپی می کند.

✓ چک لیست

□ Prototype Chain را تا null نقاشی می کنی؟

□ ۴ کار new را می گویی؟

□ کلاس را به prototype تبدیل می کنی؟

□ Composition را به جای ارث بری عمیق استفاده می کنی؟

فصل بعد: آرایه ها – که در JS اصلاً آرایه نیستند، بلکه شیء خاص هستند!

09 آرایه‌ها – فراتر از لیست – شیء خاص با ترفندها

آرایه در JS اصلاً آرایه نیست – یک شیء با کلیدهای عددی و ویژگی `length` جادویی است. اگر این را بفهمی، می‌فهمی چرا `arr[10]=100` حفره می‌سازد و چرا `toSorted` بهتر از `sort` است.

🎯 اهداف یادگیری

- ♦ توضیح دهی چرا آرایه در JS شیء است و حفره‌دار می‌شود
- ♦ جدول Mutating vs Non-Mutating را حفظ باشی و در React فقط Non-Mutating استفاده کنی
- ♦ `groupBy`, `with`, `toSpliced`, `toSorted` را با مثال بگویی
- ♦ `forEach` vs `for...in` vs `for...of` را با trade-off توضیح دهی

🧠 مدل ذهنی: آرایه = شیء + `length` جادویی

```
array-truth.js

typeof [] // "object" - !آرایه شیء است ؛
[] instanceof Array // true
Array.isArray([]) // true - (instanceof در iframe امن‌ترین چک)

const arr = [1,2,3];
arr[10] = 100; // [1,2,3, empty*7, 100] - حفره‌دار! length=11
// arr در واقع: {10:100, 2:3, 1:2, 0:1, length:11} + prototype Array

// undefined! - empty slot حفره چیست؟
[1,,3].map(x=> x*2) // [2, empty, 6] - map می‌کند!
```

⚖️ دو دسته متد – مهم‌ترین جدول این فصل

غیرجهشی (Non-mutating) – ☒	جهشی (Mutating) – ☐ در React/Redux
map, filter, slice – کپی جدید	push, pop, shift, unshift – اصلی را عوض می‌کند
<code>toSpliced</code> , <code>toSorted</code> , <code>toReversed</code> , <code>with</code> – ES2023 – کپی جدید	splice, sort, reverse – اصلی را عوض می‌کند

```
// ✗ React قدیمی - جهشی - باعث باگ رندر در
const arr = [3,1,2];
arr.sort((a,b)=>a-b); // arr اصلی عوض شد!
arr.splice(1,1); // arr اصلی عوض شد

// ✓ React غیرجهشی - امن برای - ES2023 مدرن
const sorted = arr.toSorted((a,b)=>a-b); // arr اصلی سالم، sorted جدید
const removed = arr.toSpliced(1,1); // حذف بدون جهش
const replaced = arr.with(0, 100); // جایگزینی ایندکس 0 - بدون جهش

// را جهش نده - همیشه کپی جدید state هیچ وقت آرایه/شیء React: قانون طلایی
```

array-pro.js

```
const users = [
  {id:1, name:"Ali", active:true, score:90, role:"admin"},
  {id:2, name:"Sara", active:false, score:85, role:"user"},
  {id:3, name:"Reza", active:true, score:95, role:"user"}
];

// زنجیره تمیز – خوانا
users.filter(u=>u.active).map(u=>u.name).join(", "); // "Ali, Reza"

// قدرتمندترین – هر چیزی می‌سازد – reduce
const total = users.reduce((sum,u)=>sum+u.score,0); // 275
const byId = users.reduce((acc,u)=> (acc[u.id]=u, acc), {}); // {1:{...},2:{...}}

// groupBy – ES2024 – گروه‌بندی بدون lodash
const grouped = Object.groupBy(users, u=> u.active ? "active" : "inactive");
// { active: [{...},{...}], inactive: [{...}] }

// جستجو – find, some, every
users.find(u=>u.id===2); // {id:2...}
users.some(u=>u.score>90); // true – حداقل یکی
users.every(u=>u.score>80); // true – همه

// تخت کردن – flat, flatMap
[1,[2,[3]]].flat(2); // [1,2,3] – عمق 2
[1,2,3].flatMap(x=>[x,x*2]); // [1,2,2,4,3,6] – map + flat

// ساخت آرایه از هر چیز – from
Array.from({length:5}, (_,i)=> i+1); // [1,2,3,4,5] – range
Array.from("hello"); // ["h","e","l","l","o"]
```


حلقه‌ها – کدام سریع‌تر، کدام خواناتر؟

loops-deep.js

```
// ✓ for...of – خوانا – بهترین برای آرایه + break
for (const user of users) {
  if (user.score < 80) break; // می‌توانی break کنی
  console.log(user.name);
}

// ✓ forEach – break/return وقتی نمی‌خواهی – کنی
users.forEach((u,i)=> console.log(i, u.name));
// ✗ forEach break/await نمی‌تواند!

// ✗ for...in – prototype برای آرایه هرگز! – کلیدهای رشته‌ای
for (const key in users) {} // "0","1","2" – اگر کسی + رشته!
// می‌آید!

// ✓ for – کلاسیک – سریع‌ترین – برای پرفورمنس میلیونی
for (let i=0; i<users.length; i++) {
  console.log(users[i]);
}

// در 99% موارد خوانایی مهم‌تر – ms~10 تفاوت در 1: for > for...of > forEach
// بنچمارک
```

✦ پرفورمنس – قانون

خوانایی را فدای میکرو بهینه‌سازی نکن مگر در hot path با 1M+ آیتم. for...of در 99% موارد کافی و خواناست.

تمرین فعال

تمرین ۱: آرایه [1,2,2,3,3,3] را unique کن – با Set و با filter.

تمرین ۲: با reduce یک groupBy دستی بساز – قبل از اینکه Object.groupBy را ببینی.

چالش بازار کار: یک virtual list بساز که 100k آیتم را بدون lag نمایش دهد – فقط 20 آیتم visible را رندر کن (فصل 34).

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت slice و splice؟ slice غیرجهشی و کپی برمی‌گرداند (با اندیس منفی هم کار می‌کند)؛ splice جهشی و آرایه اصلی را تغییر می‌دهد و آرایه حذف‌شده‌ها را برمی‌گرداند.

Senior – چرا toSorted بهتر از sort است؟ چون immutable است و با مدل React/Redux سازگار – sort آرایه اصلی را جهش می‌دهد و باعث باگ رندر می‌شود چون reference عوض نمی‌شود و React فکر می‌کند تغییری نکرده.

Senior – چرا Array.isArray بهتر از instanceof Array ؟ چون instanceof در iframe یا realm متفاوت false می‌دهد – هر Array iframe خودش را دارد؛ Array.isArray cross-realm امن است.

✓ چک‌لیست

- ☐ می‌توانی توضیح دهی چرا آرایه شئی است و حفره چیست؟
 - ☐ جدول Mutating vs Non-Mutating را حفظی و در React فقط Non-Mutating؟
 - ☐ toSorted , toSpliced , with , groupBy را با مثال می‌گویی؟
 - ☐ تفاوت for...of و for...in را با trade-off می‌گویی؟
- فصل بعد:** رشته‌ها – که immutable هستند و با Unicode درست کار کردن، خودش یک مهارت است.

10 رشته‌ها و Regex – از Unicode تا Template Literal

رشته در JS immutable است – هر بار که + می‌کنی، رشته جدید می‌سازی. و با Unicode درست کار کردن، خودش یک مهارت Senior است – چون `"😊".length === 2`!

🎯 اهداف یادگیری

- ◆ Template Literal و Tagged Template را با مثال SQL امن توضیح دهی
- ◆ ۵ متد مدرن رشته (`isWellFormed`, `replaceAll`, `at`) را بگویی
- ◆ Regex با Named Groups و Lookbehind بنویسی
- ◆ توضیح دهی چرا `"😊".length === 2` و راه‌حلش چیست

🧠 مدل ذهنی: رشته = آرایه‌ای از UTF-16 Code Units – نه کاراکتر!

JS رشته را با UTF-16 ذخیره می‌کند – هر کاراکتر 1 یا 2 Code Unit.

```
unicode.js

"a".length // 1 – یک Code Unit
"😊".length // 2 – چرا؟ چون 😊 خارج از BMP 2 است و Surrogate می‌خواهد!؛
[..."😊"].length // 1 – Spread با iterator – چون iterator درست می‌شمارد – نه Code Unit می‌کند، نه

// راه‌حل مدرن – ES2024:
"a\uD800".isWellFormed() // false – lone surrogate
"a\uD800".toWellFormed() // "a💩" – جایگزین با 💩

// Intl.Segmenter (ES2022) – برای طول واقعی:
const segmenter = new Intl.Segmenter("fa", { granularity: "grapheme" });
[...segmenter.segment("سلام 😊")].length // تعداد گرافیم واقعی
```

```
template-pro.js

const name = "Ali";
const msg = `Hi ${name},
You have ${2+3} messages.;
`; // بدون \n + interpolation چندخطی

// Tagged Template – DSL می‌سازد!
function highlight(strings, ...values) {
  return strings.reduce((acc, str, i) =>
    acc + str + (values[i] ? `${values[i]}` : ''), '');
}
const html = highlight`Hello ${name}, you are ${"admin"}`;
// "Hello <b>Ali</b>, you are <b>admin</b>"

// Injection – امن – جلوی SQL – کاربرد واقعی
function sql(strings, ...values) {
  const query = strings.reduce((acc, str, i) => acc + str + (values[i] ? `${values[i]}` : ''), '');
  return { query, values }; // پارامتر جدا – امن
}
const userId = 1;
const q = sql`SELECT * FROM users WHERE id = ${userId}`;
// { query: "SELECT * FROM users WHERE id = 1", values: [1] } – امن!

// Raw – بدون escape
String.raw`C:\path\to\file`; // "C:\\path\\to\\file" نه "C:\path"
```

string-pro.js

```
"hello".at(-1); // "o" – ES2022 – به جای – اندیس منفی – str[str.length-1]
" hi ".trim(); // "hi"
" hi ".trimStart(); " hi ".trimEnd();
"js".padStart(5,"*"); // "***js" – برای جدول
"js".padEnd(5,"-"); // "js---"

"ab".repeat(3); // "ababab"
"hello world".replaceAll("l","L"); // "heLLo worLd" – ES2021 – به جای /l/g

// Match – امن با Optional Chaining
"test@example.com".match(/@(.+)/)?.[1]; // "example.com" – نه undefined، نبود match اگر –
خطا
```

regex-pro.js

```
// Flags – پرچم‌ها
/re/g // global – همه
/re/i // case-insensitive
/re/m // multiline – ^ $ برای هر خط
/re/s // dotAll – . شامل \n هم می‌شود ES2018
/re/u // unicode – emoji برای
/re/v // unicodeSets – ES2024 برای \p{}
/re/y // sticky – از lastIndex شروع

// Named Groups – ES2018 – به‌جای m[1], m[2]
const re = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;
const { groups } = "2026-09-07".match(re);
groups.year // "2026" – خوانا!

// Lookbehind – ES2018 – match قبل و بعد را چک کن بدون اینکه جزو
/(?<=\$)\d+/.exec("$100")[0]; // "100" – $ بعد از – Lookbehind
/\d+(?=\$)/.exec("100$")[0]; // "100" – $ قبل از – Lookahead

// hasIndices – ES2022 – match اندیس دقیق
const m = /a(b)/d.exec("ab");
m.indices // [[0, 2], [1, 2]] – اندیس کل و گروه‌ها

// matchAll – ES2020 – همه match با groups
[..."test1 test2".matchAll(/test(\d)/g)].map(m=>m[1]); // ["1", "2"] – exec به‌جای حلقه
```

Unicode – در مصاحبه می‌پرسند ⚠️

چرا `/./v` یا `/./u` بدون `u` surrogate، flag را جدا می‌بیند؛ با `/./v` یا `/./u` درست کار می‌کند.

تمرین 🧪

تمرین ۱: تابعی بنویس `reverseString(str)` که emoji را درست برعکس کند – با `[...str].reverse().join("")`.

تمرین ۲: یک Regex بنویس که ایمیل را با Named Groups بگیرد: `(?<user>.*)(?<domain>.*)`.

چالش: یک Tagged Template `safeHtml` بساز که `<`, `>`, `&` را escape کند – جلوی XSS.

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت match و matchAll ؟ با `/g` فقط آرایه رشته‌ها برمی‌گرداند؛ `matchAll` iterator از `match objects` با `groups` و `indices` – برای `parsing`.

Senior – چرا `"😊".length === 2` ؟ چون JS UTF-16 است و emoji خارج BMP 2 Code Unit می‌خواهد؛ برای طول واقعی از `[...str].length` یا `Intl.Segmenter`.

Senior – تفاوت `/u` و `/v` ؟ `flag` `/u` : unicode mode `/v` : unicodeSets mode (ES2024) – برای `\p{}` و set operations مثل `[\p{Letter}&&\p{Script=Latin}]`.

✓ چک‌لیست

- ☐ Tagged Template برای SQL امن می‌سازی؟
 - ☐ `at(-1)` و `replaceAll` را به‌جای قدیمی استفاده می‌کنی؟
 - ☐ Regex با Named Groups و Lookbehind می‌نویسی؟
 - ☐ توضیح می‌دهی چرا `emoji length 2` است و راه‌حلش؟
- فصل بعد: `this`** – رازی که با ۴ قانون حل می‌شود.

11 this – راز بزرگ – با ۴ قانون حل می‌شود

this در JS پویا است، نه ایستا – مقدارش در زمان فراخوانی تعیین می‌شود، نه نوشتن (به جز Arrow). اگر ۴ قانون را بفهمی، دیگر هیچ وقت گیج نمی‌شوی – و باگ معروف React را هم حل می‌کنی.

🎯 اهداف یادگیری

- ♦ ۴ قانون this را با ترتیب اولویت بگویی: new > bind > implicit > default
- ♦ تفاوت Arrow و Function در this را با مثال setTimeout توضیح دهی
- ♦ ۳ راه حل گم شدن this را بگویی: call/bind/arrow wrapper
- ♦ در کلاس، چرا `inc = () =>` امن‌تر از `inc()` است را توضیح دهی

🧠 مدل ذهنی: this = کسی که تابع را صدا زده – نه جایی که نوشته شده

```
this-mental.js

function greet() { console.log(this.name); }

const ali = { name: "Ali", greet };
const sara = { name: "Sara", greet };

ali.greet(); // "Ali" – this = ali – چون ali صدا زده
sara.greet(); // "Sara" – this = sara – چون sara صدا زده
greet(); // "" یا undefined – چون کسی صدا زده – default

// مثل ضمیر "من" – هر کسی "من" می‌گوید، به خودش اشاره دارد this: تشبیه
```


this-4rules.js

```
// 1. Default – در strict mode undefined، وگرنه window
function foo() { console.log(this); }
foo(); // window (غیر strict) یا undefined (strict)

// 2. Implicit – از روی شیء قبل نقطه – obj.foo()
const obj = { name: "Ali", greet() { console.log(this.name); } };
obj.greet(); // "Ali" – this = obj – چون obj قبل نقطه است

// 3. Explicit – با call/apply/bind – تو صریح می‌گویی
function greet() { console.log(this.name); }
greet.call({name: "Sara"}); // "Sara" – call فوراً اجرا
greet.apply({name: "Reza"}); // "Reza" – apply آرایه می‌گیرد
const bound = greet.bind({name: "Bound"}); // bind تابع جدید می‌سازد – اجرا نمی‌کند
bound(); // "Bound"

// 4. new – this = شیء جدید – بالاترین اولویت
function User(name) { this.name = name; }
new User("Ali").name // "Ali" – this = {}
```

ترتیب اولویت: new < bind/call/apply < obj.foo() < default – اگر چند قانون همزمان بود، اولویت بالاتر برنده.

Arrow Function – this – از بیرون می‌گیرد

Arrow **this** خودش را ندارد – از scope بیرونی (lexical) می‌گیرد و هیچ‌وقت عوض نمی‌شود – حتی با `call/bind`.

arrow-this-deep.js

```
const obj = {
  name: "Ali",
  arrow: () => console.log(this.name), // ✗ this = window – چون arrow در global
  // نوشته شده!
  method() {
    const arrow = () => console.log(this.name); // ✓ this = obj – چون arrow در method
    // دارد this=obj نوشته شده و
    arrow();
  }
};
obj.method(); // "Ali"

// اثبات: Arrow با call عوض نمی‌شود
const arrow = () => console.log(this.name);
arrow.call({name: "Sara"}); // همچنان window – نه Sara! – چون Arrow this ندارد که عوض شود
```

lost-this.js

```
const user = {
  name: "Ali",
  greet() { console.log(this.name); }
};

const fn = user.greet; // فقط تابع جدا شد - شیء نه
fn(); // undefined - this چون - default
// در لحظه call/apply: راه حل ۱
fn.call(user); // "Ali"

// ثابت this تابع جدید با - bind: راه حل ۲
const bound = user.greet.bind(user);
bound(); // "Ali" - همیشه

// راز بیرون می‌گیرد this - Arrow wrapper: راه حل ۳
setTimeout(() => user.greet(), 100); // "Ali" - Arrow this (user) را از بیرون می‌گیرد

// مدرن‌ترین - field arrow - راه حل ۴: در کلاس
class Counter {
  count = 0;
  inc() { this.count++; } // ✗ اگر به عنوان callback پاس داده شود، this گم می‌شود
  incArrow = () => { this.count++; } // ✓ Arrow this همیشه امن - چون
  می‌گیرد
}

const c = new Counter();
setTimeout(c.inc, 100); // ✗ NaN - this=undefined
setTimeout(c.incArrow, 100); // ✓ کار می‌کند - this=c
```

```
JS

class App {
  handleClick() { console.log(this); } // this = undefined در onClick
}

<button onClick={app.handleClick}> // ✖ this شد گم!

// ✔
class App {
  handleClick = () => { console.log(this); } // Arrow field – this همیشه نمونه
}
```

تمرین

تمرین ۱: خروجی را پیش‌بینی کن:

```
JS

const obj = { name:"Ali", foo: function() { return ()=> this.name; } };
const bar = obj.foo();
bar(); // ?
bar.call({name:"Sara"}); // ?
```

تمرین ۲: یک `bind` دستی بنویس – تابعی که تابع و `this` را بگیرد و تابع جدید با `this` ثابت برگرداند.

چالش بازار کار: یک `EventEmitter` بساز که `on` و `emit` دارد و `this` در listener درست باشد – با Arrow یا `.bind`.

“ سؤالات مصاحبه (با پاسخ) ”

this – Junior چیست؟ مرجعی که تابع هنگام فراخوانی به آن وصل می‌شود؛ مقدارش بسته به نحوه فراخوانی تعیین می‌شود، نه محل تعریف – به‌جز Arrow که lexical است.

Mid – تفاوت call و apply و bind؟ `call` فوراً اجرا می‌کند؛ `bind` تابع جدید می‌سازد. `call` آرگومان‌ها را جدا می‌گیرد (`call(obj, a, b)`)، `apply` آرایه (`apply(obj, [a, b])`).

Senior – چرا Arrow Function با `this call/apply` عوض نمی‌شود؟ چون Arrow در spec `[[ThisMode]]`: lexical دارد – اصلاً `this` ندارد که `bind` شود؛ `this` را از Env بیرونی می‌خواند.

Senior – چرا در کلاس `inc = () =>` حافظه بیشتر می‌گیرد؟ چون هر نمونه یک کپی جدا از Arrow می‌سازد (روی instance) نه روی prototype – برای 1000 نمونه، 1000 تابع. ولی برای callback امن‌تر است – trade-off.

- ۴ قانون this را با اولویت می‌گویید؟
- تفاوت Arrow و Function در this را با setTimeout توضیح می‌دهی؟
- ۳ راه حل گم شدن this را می‌گویید؟
- باگ React onClick را با Arrow field حل می‌کنی؟

فصل بعد: Async – چرا JS تکنخی است ولی 1000 کار همزمان می‌کند؟

12 مقدمه Async – از Promise تا Callback Hell – چرا JS بلاک نمی‌شود

جاوااسکریپت تکنی است – یک Call Stack دارد – ولی با Event Loop و Queue، می‌تواند هزاران درخواست را بدون بلاک هندل کند. این فصل درد Callback را می‌فهمیم تا قدر Promise را بدانیم.

🎯 اهداف یادگیری

- ♦ توضیح دهی چرا JS تکنی است ولی بلاک نمی‌شود – با Event Loop
- ♦ Callback Hell را با مثال ۳ سطحی بکشی و ۳ مشکل آن را بگویی
- ♦ Promise را با ۳ حالت Pending/Fulfilled/Rejected توضیح دهی
- ♦ `any` , `race` , `allSettled` , `Promise.all` را با تفاوت fail-fast توضیح دهی

🧠 مدل ذهنی: JS = آشپز تکنی با پیشخوان

تصور کن یک آشپز (Call Stack) فقط یک غذا را همزمان می‌پزد، ولی سفارش‌های زمان‌بر (setTimeout, fetch) را به پیشخوان (Web APIs) می‌دهد و ادامه می‌دهد – وقتی غذا آماده شد، پیشخوان زنگ می‌زند (Queue) و آشپز بعداً آن را سرو می‌کند.

single-thread.js

```
console.log("1 - سفارش نان");
setTimeout(() => console.log("2 - نان آماده - بعد از 1 ثانیه"), 1000); // به پیشخوان
console.log("3 - سفارش سالاد - فوری");
// خروجی: 1, 3, 2 - نه 1, 2, 3! چون آشپز منتظر نان نمی‌ماند - غیربلاک
```

Callback Hell – ساختار تودرتوی عمیق

قبل از Promise، فقط Callback داشتیم – و کد هرم می‌شد:

```
callback-hell-deep.js

getUser(id, (err, user) => {
  if (err) return handle(err);
  getPosts(user.id, (err, posts) => {
    if (err) return handle(err);
    getComments(posts[0].id, (err, comments) => {
      if (err) return handle(err);
      console.log(comments); // سطح تو در تو! - ساختار تودرتوی عمیق 3
    });
  });
});

// مشکلات:
// 1. خوانایی فاجعه - هرم
// 2. Error handling تکراری - هر سطح if (err)
// 3. Inversion of Control - اگر آن تابع 2 بار - callback کنترل را به تابع دیگر می‌دهی - را صدا بزند، چی؟
```

Promise – قول آینده – راه حل 🤝

Promise شیئی است که قول می‌دهد آینده یک مقدار (یا خطا) را بدهد – و فقط یک‌بار قولش را می‌شکند (settle).

```
promise-deep.js

const promise = new Promise((resolve, reject) => {
  // fetch مثل async کار
  setTimeout(() => {
    if (success) resolve(data); // قول موفق
    else reject(error); // قول شکست
  }, 1000);
});

promise
  .then(data => console.log("موفق:", data)) // شد resolve وقتی
  .catch(err => console.error("شکست:", err)) // شد reject وقتی
  .finally(() => console.log("همیشه – تمیزکاری"));

// زنجیره – تخت – نه هرم!
getUser(id)
  .then(user => getPosts(user.id)) // return Promise – زنجیره
  .then(posts => getComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(err => handle(err)); // برای همه catch یک!
```

Pending

در انتظار – قول داده ولی هنوز نه
اولیه



Fulfilled

resolve(value)
موفق – مقدار دارد



Rejected

reject(reason)
شکست – خطا دارد

قانون طلایی: Promise فقط یک‌بار settle می‌شود – بعد از آن تغییر نمی‌کند (immutable) – مثل شکستن قول – دیگر نمی‌توانی پس بگیری.


```
promise-api-deep.js

// ساخت فوری
Promise.resolve(42); // Promise برای تست
Promise.reject("err"); // Promise شکست فوری

// ترکیب - مهم‌ترین
Promise.all([p1,p2,p3]); // fail-fast - یکی شکست → همان خطا، [v1,v2,v3] → همه موفق
Promise.allSettled([p1,p2,p3]); // batch برای → همیشه موفق → [{status:"fulfilled",value:v1}, {status:"rejected",reason:e}]
Promise.race([p1,p2]); // timeout برای - (موفق یا شکست) settle اولین
Promise.any([p1,p2]); // AggregateError → اگر همه شکست - ES2021 - اولین موفق

// Timeout - مثال واقعی
function withTimeout(promise, ms) {
  const timeout = new Promise( (_,rej) => setTimeout(() => rej(new Error("Timeout")), ms));
  return Promise.race([promise, timeout]); // هر کدام زودتر
}
await withTimeout(fetch('/api'), 3000); // اگر 3 ثانیه طول کشید، خطا
```

حرفه‌ای - all vs allSettled

`Promise.all` اگر یکی شکست بخورد، بقیه را کنسل نمی‌کند - فقط نتیجه را زودتر برمی‌گرداند - ولی `Promise` های دیگر در پس‌زمینه ادامه دارند! برای کنسل واقعی از `AbortController` استفاده کن (فصل 15).

تمرین فعال

تمرین ۱: یک `delay(ms)` بنویس که `Promise` برمی‌گرداند و بعد از `ms` resolve می‌شود - با `new Promise`.

تمرین ۲: ۳ `Promise` بساز که ۲ تا موفق و ۱ شکست - با `all`, `allSettled`, `race`, `any` تست کن و خروجی را پیش‌بینی کن.

چالش بازار کار: یک `retry(fn, times)` بنویس که `fn` (که `Promise` برمی‌گرداند) را تا `times` بار `retry` کند - با `exponential backoff`.

“ سؤالات مصاحبه (با پاسخ)

Junior – Promise چیست؟ شیئی که نتیجه آینده یک عملیات async را نمایندگی می‌کند – با 3 حالت Pending/Fulfilled/Rejected.

Mid – تفاوت race و any ؟ اولین settle (حتی reject) را برمی‌گرداند؛ any اولین fulfill را، و اگر همه reject شدند AggregateError می‌دهد – any برای "اولین موفق" بهتر است.

Senior – چرا Promise.all بقیه را کنسل نمی‌کند؟ چون Promise‌ها از قبل شروع شده‌اند – all فقط نتیجه را زودتر برمی‌گرداند؛ برای کنسل واقعی باید AbortController را به fetch پاس دهی و abort() کنی.

Senior – تفاوت Promise و Callback در Inversion of Control؟ در Callback کنترل را به تابع دیگر می‌دهی – اگر آن تابع 2 بار callback را صدا بزند یا اصلاً صدا نزند، باگ – در Promise، تو کنترل را نگه می‌داری و فقط یک‌بار settle می‌شود – امن‌تر.

✓ چک‌لیست

- ☐ می‌توانی با تشبیه آشپز، تکنخی ولی غیربلاک را توضیح دهی؟
 - ☐ Callback Hell را با ۳ مشکلش می‌گویی؟
 - ☐ ۳ حالت Promise و قانون immutable بودن را می‌گویی؟
 - ☐ تفاوت all , allSettled , race , any را با fail-fast توضیح می‌دهی؟
- فصل بعد:** وارد بخش دوم می‌شویم – JS مدرن – از Destructuring تا Event Loop عمیق.

13 Rest و Destructuring، Spread

سه نقطه ... کوچک، اما سه معنای بزرگ.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تک‌نخی با پیشخوان سفارش‌ها.

Destructuring عمیق

destructure-deep.js

```
const user = { id:1, name:"Ali", address:{ city:"Tehran" }, scores:[90,85,95] };
const { name, address:{ city }, scores:[firstScore, ...restScores], role="user" } =
user;
```

rest-spread.js

```
const [a, ...rest] = [1,2,3,4];
function sum(...nums) { return nums.reduce((s,n)=>s+n,0); }
const arr2 = [...[1,2], 3, 4];
const obj2 = { ...{x:1}, y:2 };
const defaults = { theme:"light" };
const config = { ...defaults, ...userConfig };
```

ترفندها

tricks.js

```
const { password, ...safeUser } = user;
[a,b] = [b,a];
function fetchUser({ id, cache=true, timeout=5000 }={}) {}
```

!Shallow Copy ⚠

... فقط سطح اول را کپی می‌کند. برای deep از `structuredClone`.

“سؤالات مصاحبه (با پاسخ)”

Mid – تفاوت Rest و Spread؟ Rest جمع می‌کند، Spread باز می‌کند.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟

♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت هاب پوش کن. این رزومه تو می شود.

✓ | چک لیست فهم عمیق

- ☐ می توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

for...of چگونه کار می‌کند؟ با پروتکل Iterator.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

Iterator Protocol

iterator.js

```
const arr = [1,2,3];
const iter = arr[Symbol.iterator]();
iter.next(); // {value:1, done:false}

const range = {
  from:1, to:5;;
  [Symbol.iterator]() {
    let current = this.from;
    return { next: () => current <= this.to ? { value: current++, done: false } : {
done: true } };
  }
};
[...range]; // [1,2,3,4,5]
```

Generator

generator.js

```
function* gen() { yield 1; yield 2; yield 3; }
const g = gen();
g.next();

function* rangeGen(from,to) { for (let i=from; i<=to; i++) yield i; }

function* infinite() { let i=0; while(true) yield i++; }
```

کاربردها

gen-uses.js

```
function* trafficLight() { while(true) { yield "green"; yield "yellow"; yield
"red"; } }

function* flatten(arr) {
  for (const item of arr) {
    if (Array.isArray(item)) yield* flatten(item);
    else yield item;
  }
}
```

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

15 Promise پیشرفته و Async/Await

async/await فقط قند روی Promise نیست؛ درک عمیق آن تفاوت Junior و Senior است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تک‌نخی با پیشخوان سفارش‌ها.

Async/Await زیر کاپوت

async.js

```
async function getData() {
  const user = await fetchUser();
  const posts = await fetchPosts(user.id);
  return posts;
}
```

الگوهای مهم

patterns.js

```
// سریال vs موازی
const [a2,b2] = await Promise.all([fetchA(), fetchB()]);

// Timeout
function withTimeout(promise, ms) {
  const timeout = new Promise( (_,rej) => setTimeout(() => rej(new Error("Timeout")), ms));
  return Promise.race([promise, timeout]);
}

// Retry
async function retry(fn, times=3) {
  for (let i=0; i<times; i++) {
    try { return await fn(); }
    catch (e) { if (i===times-1) throw e; }
  }
}
```

AbortController

abort.js

```
const controller = new AbortController();
fetch('/api/data', { signal: controller.signal });
controller.abort();
```

✖ اشتباه رایج

JS

```
users.forEach(async (u) => { await save(u); }); // ✖
// ✔
for (const u of users) { await save(u); }
await Promise.all(users.map(u => save(u)));
```

“سؤالات مصاحبه (با پاسخ)”

Mid – چرا `forEach` با `async` کار نمی‌کند؟ چون مقدار برگشتی `callback` را نادیده می‌گیرد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

16 Event Loop و مدل همزمانی

اگر Event Loop را بفهمی، می‌فهمی چرا `setTimeout(0)` بعد از `Promise` اجرا می‌شود.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

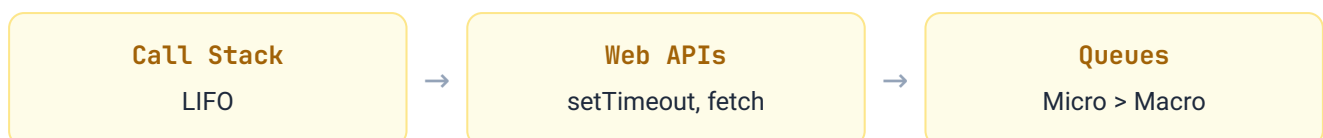
🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

معماری



ترتیب اجرا

order.js

```
console.log("1");
setTimeout(()=> console.log("2"), 0);
Promise.resolve().then(()=> console.log("3"));
queueMicrotask(()=> console.log("4"));
console.log("5"); // 1,5,3,4,2
```

Node.js Event Loop

node-loop.js

```
setTimeout(()=> console.log("timeout"), 0);
setImmediate(()=> console.log("immediate"));
```

Starvation

starvation.js

```
function loop() { Promise.resolve().then(loop); }
loop();
setTimeout(()=> console.log("never"), 0); // هیچ وقت!
```

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت Microtask و Macrotask؟ Microtask بعد از هر Stack و قبل از رندر؛ Macrotask هر بار یکی.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

17 ماژول‌ها – ESM و CJS عمیق

ماژول‌ها فقط import نیستند؛ درک CJS، ESM و بهینه‌سازی مرز Junior و Senior است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تک‌نخی با پیشخوان سفارش‌ها.

ESM vs CJS

CJS	ESM	ویژگی
require	import/export	Syntax
✗	✓	Tree-shaking
✗	✓	Top-level await

esm.mjs

```
export const x = 1;
export default function() {}
import def, { x } from './mod.js';
```

Dynamic Import

dynamic.js

```
const { default: Chart } = await import('./chart.js');
if (user.isAdmin) {
  const { AdminPanel } = await import('./admin.js');
}
```

Tree Shaking

tree.js

```
export const used = () => {};
export const unused = () => {}; // import حذف می‌شود اگر نشود
```

package.json مدرن

JSON

```
{ "type": "module", "sideEffects": false }
```

“ سؤالات مصاحبه (با پاسخ)

Mid – چرا ESM برای Tree Shaking لازم است؟ چون استاتیک و قابل تحلیل است.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

18 کلاس‌ها و OOP در JS

کلاس در JS فقط Syntax Sugar روی Prototype است، اما با ویژگی‌های مدرن ۲۰۲۴ قدرتمند شده.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

class-modern.js

```
class User {
  static #counter = 0;
  #password;
  constructor(name, password) { this.name = name; this.#password = password;
  User.#counter++; }
  get maskedPassword() { return "*".repeat(this.#password.length); }
  set password(val) { if (val.length < 8) throw new Error("Too short");
  this.#password = val; }
  greet() { return `Hi ${this.name}`; }
  static get count() { return User.#counter; }
}
```

inheritance.js

```
class Admin extends User {
  constructor(name, password, level) { super(name, password); this.level = level; }
  greet() { return `${super.greet()} [Admin ${this.level}]`; }
}
```

composition.js

```
const canFly = (obj) => ({ ...obj, fly() { console.log("flying"); } });
const canEat = (obj) => ({ ...obj, eat() { console.log("eating"); } });
const bird = canFly(canEat({ name: "Parrot" }));
```

Composition over Inheritance – تا جایی که می‌توانی از ترکیب استفاده کن.

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت #private و #private_؟ واقعاً خصوصی؛ _ فقط قرارداد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

19 مدیریت خطا – حرفه‌ای

کد بدون خطا وجود ندارد؛ کد حرفه‌ای کدی است که خطا را درست مدیریت می‌کند.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

انواع خطا

errors.js

```
throw new Error("generic");
throw new TypeError("type wrong");
try { lowLevel(); } catch (err) { throw new Error("High level", { cause: err }); }
```

try-catch.js

```
try { risky(); } catch (err) { if (err instanceof TypeError) {} else throw err; }
finally { cleanup(); }

async function fetchData() {
  try { const res = await fetch('/api'); if (!res.ok) throw new Error(`HTTP $
{res.status}`); return await res.json(); }
  catch (err) { console.error(err.cause ?? err); throw err; }
}
```

الگوهای Production

prod-errors.js

```
function tryCatch(fn) { try { return [null, fn()]; } catch (e) { return [e, null]; } }

class AppError extends Error {
  constructor(message, { status=500, code="UNKNOWN" }={}) { super(message);
  this.status=status; this.code=code; this.name="AppError"; }
}
```

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت "throw error" و "throw new Error"؟ رشته stack ندارد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.

♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟

♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت هاب پوش کن. این رزومه تو می شود.

✓ | چک لیست فهم عمیق

- ☐ می توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

20 کالکشن‌های پیشرفته

Object برای همه چیز مناسب نیست. Map و Set برای کارایی و امنیت ساخته شده‌اند.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکی با پیشخوان سفارش‌ها.

Map vs Object

ویژگی	Map	Object
کلید	هر نوع	فقط string/symbol
اندازه	size	keys().length

map.js

```
const map = new Map();
map.set({id:1}, "Ali");
const keyObj = {id:1};
map.set(keyObj, "Ali");
map.get(keyObj);
```

Set

set.js

```
const set = new Set([1,2,2,3]); // {1,2,3}
const unique = [...new Set([1,2,2,3])];
const a = new Set([1,2,3]); const b = new Set([2,3,4]);
a.intersection(b); // {2,3}
```

WeakMap

weak.js

```
const cache = new WeakMap();
function getData(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const data = heavyCompute(obj);
  cache.set(obj, data);
  return data;
}
```

“سؤالات مصاحبه (با پاسخ)”

Mid – چرا Map بهتر از Object برای کلید شیء؟ Object کلید را به رشته تبدیل می‌کند.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

21 Proxy و Reflect – متaprogramming

Proxy به تو اجازه می‌دهد رفتار پیش‌فرض شیء را بازنویسی کنی – پایه Vue 3 Reactivity.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

Proxy چیست؟

proxy.js

```
const target = { name: "Ali", age: 25 };
const proxy = new Proxy(target, {
  get(t, prop, receiver) { console.log(`get ${String(prop)}`); return Reflect.get(t, prop, receiver); },
  set(t, prop, value, receiver) { console.log(`set ${String(prop)}=${String(value)}`); return Reflect.set(t, prop, value, receiver); }
});
proxy.name; // log
```

proxy-uses.js

```
const validated = new Proxy({}, {
  set(t, p, v) { if (p === "age" && (typeof v !== "number" || v < 0)) throw new
  TypeError("Invalid age"); return Reflect.set(t,p,v); }
});

const negArray = new Proxy([1,2,3], {
  get(t,p) { const idx = Number(p); if (idx < 0) return t[t.length + idx]; return
  Reflect.get(t,p); }
});
negArray[-1]; // 3

function reactive(obj) {
  return new Proxy(obj, {
    get(t,p,recv) { track(t,p); return Reflect.get(t,p,recv); },
    set(t,p,v,recv) { const res = Reflect.set(t,p,v,recv); trigger(t,p); return res; }
  });
}
```

پرفورمنس ⚠

Proxy کندتر از دسترسی مستقیم است؛ فقط جایی که متaproگرامینگ لازم است.

“سؤالات مصاحبه (با پاسخ)”

Senior – چرا 3 از Vue Proxy استفاده می‌کند؟ چون آرایه و ویژگی جدید را هم intercept می‌کند.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

مدل ذهنی این فصل چیست؟ آن را نقاشی کن.

- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

22 برنامه‌نویسی فانکشنال در JS

فانکشنال کدت را قابل تست، قابل پیش‌بینی و بدون باگ می‌کند.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

Pure Function

pure.js

```
let count = 0;
function inc() { count++; return count; } // ✗ Impure
function incPure(count) { return count + 1; } // ✓ Pure
```

Immutability

```
immutable-fp.js

const user = { name:"Ali", age:25 };
const user2 = { ...user, age: 26 };
```

Pipe و Composition

```
composition.js

const pipe = (...fns) => (x) => fns.reduce((v,f)=> f(v), x);
const toUpper = s => s.toUpperCase();
const exclaim = s => s + "!";
const greet = s => `Hello ${s}`;
const shoutGreet = pipe(greet, toUpper, exclaim);
shoutGreet("ali"); // "HELLO ALI!"
```

Currying

```
currying.js

const curry = (fn) => {
  return function curried(...args) {
    if (args.length >= fn.length) return fn(...args);
    return (...next) => curried(...args, ...next);
  };
};

const add = (a,b,c) => a+b+c;
const curriedAdd = curry(add);
curriedAdd(1)(2)(3); // 6
```

چرا FP در JS؟

چون React، Redux، RxJS همه FP هستند.

“سؤالات مصاحبه (با پاسخ)”

Pure Function – Mid چیست؟ تابعی که خروجی فقط به ورودی بستگی دارد و side effect ندارد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

23 DOM و BOM – جاوااسکریپت در مرورگر

حتی اگر با React کار می‌کنی، باید DOM واقعی را بفهمی.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

DOM چیست؟

dom.js

```
const el = document.querySelector("#app");
const div = document.createElement("div");
div.textContent = "Hello";
el.appendChild(div);
el.append(div, "text", document.createElement("span"));
```

Capturing و Event – Bubbling

event.js

```
el.addEventListener("click", handler, { capture: false });
document.querySelector("#list").addEventListener("click", (e) => {
  if (e.target.matches(".item")) console.log("clicked", e.target.dataset.id);
});
e.stopPropagation();
```

BOM

bom.js

```
window.innerWidth; window.scrollTo();
history.pushState({}, "", "/new-url");
localStorage.setItem("key", "value");
document.cookie = "token=abc; Max-Age=3600; Secure; SameSite=Strict";
```

Performance DOM

perf-dom.js

```
const frag = document.createDocumentFragment();
for (let i=0; i<1000; i++) { const div = document.createElement("div");
div.textContent = i; frag.appendChild(div); }
el.appendChild(frag);
```

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت innerHTML و textContent؟ HTML innerHTML را parse و XSS خطر دارد؛ textContent فقط متن.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

24 Fetch، Storage و API‌های مرورگر

مرورگر یک سیستم‌عامل کوچک است — با ده‌ها API قدرتمند.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تک‌نخی با پیشخوان سفارش‌ها.

Fetch مدرن

fetch.js

```
const res = await fetch('/api/users');
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();

await fetch('/api/users', { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ name: "Ali" }), signal: AbortSignal.timeout(5000) });
```

Storage ها

Storage	حجم	ماندگاری
localStorage	5MB	دائمی
IndexedDB	تا 60% دیسک	دائمی

```
storage.js

import { openDB } from 'idb';
const db = await openDB('my-db', 1, { upgrade(db) { db.createObjectStore('users', {
keyPath: 'id' }); } });
await db.put('users', { id:1, name:"Ali" });
```

API های جذاب

```
apis.js

await navigator.clipboard.writeText("copied!");
const worker = new Worker('worker.js');
const bc = new BroadcastChannel('app');
bc.postMessage({ type: "LOGOUT" });
```

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت localStorage و IndexedDB؟ localStorage sync و 5MB؛ IndexedDB async و حجم زیاد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.

- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

25 پرفورمنس و بهینه‌سازی

کد درست کار می‌کند؛ کد سریع پول می‌سازد.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

۷۸ چگونه بهینه می‌کند؟

v8.js

```
function add(a,b) { return a+b; }
add(1,2); add(3,4); // سریع - number همیشه

const o1 = { x:1, y:2 };
const o2 = { x:3, y:4 }; // سریع - hidden class همان
const o3 = { y:5, x:6 }; // جدید - کند hidden class
```


bench.js

```
const start = performance.now();
for (let i=0; i<1e6; i++) {}
console.log(performance.now() - start);
```

opt.js

```
function debounce(fn, delay) {
  let timer;
  return (...args) => { clearTimeout(timer); timer = setTimeout(()=> fn(...args),
    delay); };
}

const memo = (fn) => {
  const cache = new Map();
  return (...args) => { const key = JSON.stringify(args); if (cache.has(key)) return
    cache.get(key); const res = fn(...args); cache.set(key,res); return res; };
};
```

“ سؤالات مصاحبه (با پاسخ)

Senior – تفاوت Debounce و Throttle؟ Debounce بعد از سکوت؛ Throttle هر X ms.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟

♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت هاب پوش کن. این رزومه تو می شود.

✓ | چک لیست فهم عمیق

- ☐ می توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

26 حافظه و Garbage Collection

JS حافظه را خودکار مدیریت می‌کند، اما اگر نفهمی چگونه، Memory Leak می‌گیری.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکی با پیشخوان سفارش‌ها.

Stack vs Heap

- ♦ Stack: Primitive ها، reference ها – سریع
- ♦ Heap: اشیا، آرایه‌ها – کندتر، GC می‌شود

stack-heap.js

```
let a = 10; // Stack
let obj = { x:10 }; // reference در Stack، {x:10} در Heap
```

gc.js

```
let user = { name: "Ali" };
let admin = user;
user = null; // هنوز یک reference
admin = null; // GC قابل reference حالا 0
```

weakref.js

```
let obj = { data: "big" };
const weak = new WeakRef(obj);
obj = null;
console.log(weak.deref()); // undefined اگر GC شده
```

“سؤالات مصاحبه (با پاسخ)

Senior – چرا WeakMap مانع GC نمی‌شود؟ چون reference ضعیف نگه می‌دارد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

27 امنیت در جاوااسکریپت

امنیت یک فیچر نیست، یک پیش‌فرض است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

XSS

xss.js

```
el.innerHTML = userInput; // ✗
el.textContent = userInput; // ✔
import DOMPurify from 'dompurify';
el.innerHTML = DOMPurify.sanitize(userInput); // ✔
eval(userInput); // ✗ هرگز!
```

Prototype Pollution

pollution.js

```
const payload = JSON.parse('{"__proto__": {"isAdmin": true}}');
_.merge({}, payload);
console.log({}.isAdmin); // true - فاجعه!

function isSafeKey(key) { return !
["__proto__", "constructor", "prototype"].includes(key); }
```

CORS و CSRF

csrf.js

```
document.cookie = "token=abc; SameSite=Strict; Secure";
fetch('/api', { credentials: "include" });
```

✖ چک لیست امنیت

- ☐ هیچ وقت innerHTML با ورودی کاربر بدون sanitize
- ☐ هیچ وقت eval
- ☐ کوکی HttpOnly, Secure, SameSite=Strict

“ سؤالات مصاحبه (با پاسخ)

XSS – Mid چیست؟ تزریق اسکریپت مخرب از طریق ورودی کاربر.

🔧 تمرین فعال – برای جا افتادن عمیق

♦ تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.

- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

28 تست‌نویسی مدرن

کد بدون تست، بدهی فنی است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

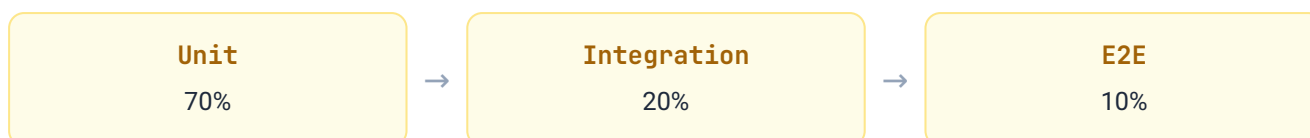
🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکی با پیشخوان سفارش‌ها.

هرم تست



```

math.test.js

import { describe, it, expect, vi } from 'vitest';
import { sum } from './math.js';

describe('sum', () => {
  it('adds numbers', () => { expect(sum(1,2)).toBe(3); });
});
vi.mock('./api.js', () => ({ fetchUser: vi.fn().mockResolvedValue({ id:1 }) }));

```

```

e2e.spec.js

import { test, expect } from '@playwright/test';
test('login flow', async ({ page }) => {
  await page.goto('/login');
  await page.fill('input[name="email"]', 'test@example.com');
  await page.click('button[type="submit"]');
  await expect(page).toHaveURL('/dashboard');
});

```

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت Unit و Integration؟ Unit یک واحد جدا؛ Integration تعامل چند واحد.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.

♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟

♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت هاب پوش کن. این رزومه تو می شود.

✓ | چک لیست فهم عمیق

- ☐ می توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

29 ابزارها و باندلرها

ابزار درست، 10 برابر بهره‌وری می‌دهد.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

Vite

vite.config.js

```
import { defineConfig } from 'vite';
export default defineConfig({
  server: { port: 3000, open: true },
  build: { target: "es2023", minify: "esbuild" }
});
```

ESLint 9 Flat Config

eslint.config.js

```
import js from '@eslint/js';
export default [js.configs.recommended, { rules: { "no-var": "error", "prefer-const": "error" } }];
```

Biome

biome.json

```
{
  "linter": { "enabled": true },
  "formatter": { "enabled": true, "indentWidth": 2 }
}
```

چرا Vite؟

چون در Dev از ESM نیکیو مرورگر استفاده می‌کند – بدون باندل!

“سؤالات مصاحبه (با پاسخ)”

Senior – تفاوت Vite و Webpack؟ Vite در Dev باندل نمی‌کند.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

30 TypeScript و تعامل با JS

در ۲۰۲۶ نوشتن JS بدون TS مثل رانندگی بدون کمربند است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

چرا TS؟

ts.ts

```
type User = { id:number; name:string; email?:string; readonly createdAt:Date; };
function greet(user: User): string { return `Hi ${user.name}`; }
```

```
jsdoc.js

// @ts-check
/**
 * @param {string} name
 * @param {number} [age]
 * @returns {{ name: string, age: number }}
 */
function createUser(name, age=18) { return { name, age }; }
```

```
utils.ts

type User = { id:number; name:string; email:string; password:string };
type PublicUser = Pick<User, "id" | "name">;
type SafeUser = Omit<User, "password">;
type PartialUser = Partial<User>;
type R = ReturnType<(a:number)=> boolean>;
```

“ سؤالات مصاحبه (با پاسخ)

Mid – تفاوت any و unknown؟ TS any را خاموش؛ unknown امن.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

31 الگوها و معماری تمیز

الگوها راه‌حل‌های اثبات‌شده برای مشکلات تکراری هستند.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

patterns.js

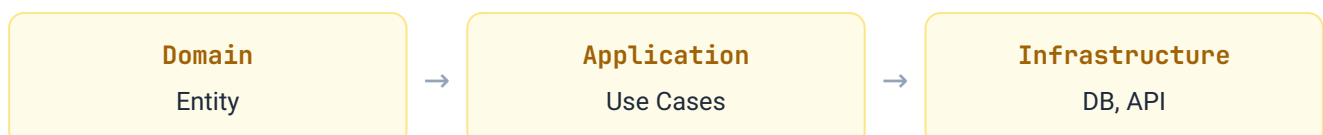
```
class DB {
  static #instance;
  constructor() { if (DB.#instance) return DB.#instance; DB.#instance = this;
  this.connection = "connected"; }
}
const createUser = (type) => { if (type=="admin") return new Admin(); return new User(); };

class QueryBuilder {
  #query = {};
  select(fields) { this.#query.select = fields; return this; }
  where(cond) { this.#query.where = cond; return this; }
  build() { return this.#query; }
}
```

behavioral.js

```
class EventEmitter {
  #events = new Map();
  on(event, fn) { if (!this.#events.has(event)) this.#events.set(event, []);
  this.#events.get(event).push(fn); }
  emit(event, ...args) { this.#events.get(event)?.forEach(fn=> fn(...args)); }
}

const strategies = { credit: (a) => a*1.02, paypal: (a) => a*1.03 };
const pay = (method, amount) => strategies[method](amount);
```



clean.js

```
export class User {
  constructor(name, email) { if (!email.includes("@")) throw new Error("Invalid email"); this.name=name; this.email=email; }
}
export const createUser = async ({ userRepo }, { name, email }) => {
  const user = new User(name, email); await userRepo.save(user); return user;
};
```

“ سؤالات مصاحبه (با پاسخ)

Senior – تفاوت Singleton و Module؟ Module خودش Singleton است.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟

□ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

32 متaproگرامینگ پیشرفته

کدی که کد می‌نویسد — متaproگرامینگ مرز بین کتابخانه و فریم‌ورک است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

Symbol

symbol.js

```
const sym1 = Symbol("desc");
const sym2 = Symbol("desc");
sym1 === sym2 // false

const obj = {
  [Symbol.iterator]: function*() { yield 1; yield 2; },
  get [Symbol.toStringTag]() { return "MyObj"; }
};
Object.prototype.toString.call(obj); // "[object MyObj]"
```

decorator.js

```
function logged(target, context) {
  return class extends target { constructor(...args) { console.log(`Creating $
{context.name}`); super(...args); } };
}
@logged
class User { constructor(name) { this.name=name; } }

function memo(target, context) {
  const cache = new Map();
  return function(...args) { const key = JSON.stringify(args); if (cache.has(key))
return cache.get(key); const res = target.apply(this, args); cache.set(key,res);
return res; };
}
class Calculator { @memo fib(n) { return n<=1 ? n : this.fib(n-1)+this.fib(n-2); } }
```

Tagged Templates

dsl.js

```
function sql(strings, ...values) {
  const query = strings.reduce((acc,str,i)=> acc+str+(values[i] ? `$$${i+1}` : ''),
  '');
  return { query, values };
}
const q = sql`SELECT * FROM users WHERE id = ${1}`;
```

“ سؤالات مصاحبه (با پاسخ)

Senior – Decorator چیست؟ تابعی که کلاس/متد را wrap و رفتار اضافه می‌دهد.

تمرین فعال — برای جا افتادن عمیق

تمرین ۱ — Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی — بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای — دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

33 نکته و ترفند طلایی

نکات کوچک، تفاوت Junior و Senior را می‌سازد.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

۱. Nullish Coalescing

JS

```
const count = input ?? 10;
```

۲. Rename با Destructuring

JS

```
const { name: userName, age=18 } = user;
```

۳. حذف ویژگی بدون Mutation

JS

```
const { password, ...safe } = user;
```

۴. تبدیل به Boolean با !!

JS

```
!!"hello" // true
```

۵. Swap

JS

```
[a,b] = [b,a];
```

۶. Unique آرایه

JS

```
[...new Set(arr)]
```

۷. GroupBy جدید

JS

```
Object.groupBy(users, u=> u.role);
```

۸. at(-1)

JS

```
arr.at(-1)
```

۹. Promise.all موازی

JS

```
const [user, posts] = await Promise.all([fetchUser(), fetchPosts()]);
```

۱۰. AbortSignal.timeout

JS

```
fetch(url, { signal: AbortSignal.timeout(3000) });
```

۱۱. structuredClone

JS

```
const deep = structuredClone(obj);
```

۱۲. Proxy Validation

JS

```
new Proxy(obj, { set(t,p,v){ if(p==="age" && v<0) throw Error(); return Reflect.set(t,p,v); }});
```

۱۳. Currying

JS

```
const multiply = a=> b=> a*b;  
const double = multiply(2);
```

۱۴. Pipe

JS

```
const pipe = (...fns) => x => fns.reduce((v,f)=> f(v), x);
```

Optional Chaining .۱۵

JS

```
user?.address?.city ?? "نامشخص"
```

Numeric Separator .۱۶

JS

```
1_000_000
```

Object.hasOwn .۱۷

JS

```
Object.hasOwn(obj, "key")
```

Array.from Range .۱۸

JS

```
Array.from({length:5}, (_,i)=> i+1)
```

toSorted Immutable .۱۹

JS

```
arr.toSorted((a,b)=> a-b)
```

Intl فرمت فارسی .۲۰

JS

```
new Intl.NumberFormat("fa-IR").format(1000000) // "۱,۰۰۰,۰۰۰"
```

۲۱. crypto.randomUUID

JS

```
crypto.randomUUID()
```

۲۲. WeakMap Private

JS

```
const priv = new WeakMap();
```

۲۳. Generator Lazy

JS

```
function* gen() { let i=0; while(true) yield i++; }
```

۲۴. Early Return

early-return.js

```
function getUser(user) {  
  if (!user) return null;  
  return user.name;  
}
```

✦ حرفه‌ای

این نکات را چاپ کن و کنار مانیتور بچسبان.

🔧 تمرین فعال – برای جا افتادن عمیق

✦ تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ | چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

♦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 | اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

یادگیری بدون پروژه، فراموشی است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تک‌نخی با پیشخوان سفارش‌ها.

پروژه ۱: Debounce Search

debounce.js

```
function debounce(fn, delay) {
  let timer;
  return (...args) => { clearTimeout(timer); timer = setTimeout(() => fn(...args), delay); };
}

const input = document.querySelector("#search");
input.addEventListener("input", debounce(async (e) => { const results = await fetch(`/api/search?q=${e.target.value}`).then(r => r.json()); render(results); }, 300));
```

پروژه ۲: Memoization

memo.js

```
function memo(fn, { max=100 }={}) {
  const cache = new Map();
  return (...args) => { const key = JSON.stringify(args); if (cache.has(key)) return
cache.get(key); const res = fn(...args); if (cache.size >= max)
cache.delete(cache.keys().next().value); cache.set(key,res); return res; };
}
```

پروژه ۳: EventEmitter

emitter.js

```
class EventEmitter {
  #events = new Map();
  on(e,fn){ (this.#events.get(e) ?? this.#events.set(e,[]).get(e)).push(fn); return
()=> this.off(e,fn); }
  off(e,fn){ const arr=this.#events.get(e); if(arr) this.#events.set(e,
arr.filter(f=>f!==fn)); }
  emit(e,...args){ this.#events.get(e)?.forEach(fn=>fn(...args)); }
}
```

پروژه ۴: Promise Pool

pool.js

```
async function pool(tasks, concurrency=3) {
  const results = []; const executing = new Set();
  for (const task of tasks) {
    const p = task().then(res=> { executing.delete(p); return res; });
    executing.add(p); results.push(p);
    if (executing.size >= concurrency) await Promise.race(executing);
  }
  return Promise.all(results);
}
```


پروژه ۵: Virtual List

virtual.js

```
function renderVirtual({ container, items, itemHeight, visibleCount }) {  
  let start = 0;  
  container.addEventListener("scroll", ()=>{ start = Math.floor(container.scrollTop /  
itemHeight); const end = Math.min(start+visibleCount, items.length);  
container.innerHTML = items.slice(start,end).map(i=> `<div style="height:${itemHeight}  
px">${i}</div>`).join(""); });  
}
```

پروژه ۶: Proxy با Undo/Redo

undo.js

```
function withHistory(obj) {  
  const history = [structuredClone(obj)]; let index = 0;  
  const proxy = new Proxy(obj, { set(t,p,v) { const res = Reflect.set(t,p,v);  
history.splice(index+1); history.push(structuredClone(t)); index++; return res; } });  
  return { state: proxy, undo() { if(index>0) { index--; Object.assign(proxy,  
history[index]); } }, redo() { if(index<history.length-1) { index++;  
Object.assign(proxy, history[index]); } } };  
}
```

پروژه ۷: Retry با Fetch

fetch-pro.js

```
const cache = new Map();  
async function smartFetch(url, { retries=3, ttl=60000 }={}) {  
  const cached = cache.get(url);  
  if (cached && Date.now()-cached.time < ttl) return cached.data;  
  for (let i=0; i<retries; i++) {  
    try { const res = await fetch(url, { signal: AbortSignal.timeout(5000) }); if (!  
res.ok) throw new Error(`HTTP ${res.status}`); const data = await res.json();  
cache.set(url, { data, time: Date.now() }); return data; }  
    catch (e) { if (i===retries-1) throw e; await new Promise(r=> setTimeout(r,  
1000*2**i)); }  
  }  
}
```

پروژه ۸: Kanban

kanban.js

```
const board = { todo: [{id:1, title:"Task 1"}], doing: [], done: [] };  
function move(taskId, from, to) { const idx = board[from].findIndex(t=>  
t.id===taskId); if (idx===-1) return; const [task] = board[from].splice(idx,1);  
board[to].push(task); save(board); render(board); }
```

➤ چالش اضافه

هر پروژه را با تست Vitest و TypeScript بازنویسی کن.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

35 ۲۰ اشتباه رایج و راه حل

این ۲۰ اشتباه را هر JS Developer حداقل یک بار مرتکب شده.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

#	اشتباه	راه حل
1	== به جای ===	همیشه ===
2	var	فقط let/const
3	فراموشی await در forEach	for...of یا Promise.all(map)
4	sort با Mutation	toSorted
5	innerHTML با ورودی کاربر	textContent یا DOMPurify
6	عدم cleanup EventListener	removeEventListener
7	setInterval بدون clear	همیشه clear
8	JSON.parse بدون try	try در wrap
9	مقایسه اشیا با ===	deepEqual
10	this گم شده	bind یا Arrow

=== ✓

JS

```
"0" === 0 // false
```

== ✗

JS

```
"0" == 0 // true - باگ!
```

for...of ✓

JS

```
for (const u of users) { await save(u); }
```

forEach async ✗

JS

```
users.forEach(async u=>{ await save(u); });
```

اشتباهات پیشرفته

Prototype Pollution

JS

```
_.merge({}, JSON.parse('{"__proto__":{"polluted":true}}'))
// ✓ Object.create(null)
```

```
JS

function leak() { const big = new Array(1e6); return () => big[0]; }
// ✓ نگه دار first فقط
```

```
JS

fetch('/api'); // ✗ unhandledRejection
fetch('/api').catch(console.error); // ✓
```

✗ چک لیست Code Review

- ☐ همه == به === ؟
- ☐ همه EventListener ها cleanup ؟
- ☐ همه Promise ها handle ؟

“ سؤالات مصاحبه (با پاسخ)

Senior – چرا Floating Promise خطرناک است؟ چون خطا ساکت می ماند.

🔧 تمرین فعال – برای جا افتادن عمیق

♦ تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ ۲ اشتباه رایج این مبحث چیست و راه حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

36 ۵۰ پرسش مصاحبه – از Junior تا Senior

مصاحبه JS فقط سوال نیست – مدل ذهنی می‌خواهد. ۵۰ پرسش FAANG.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

Junior (۱-۱۰)

۱. تفاوت **let** و **var**؟ **var** function-scope و **let** با **hoist** و **TDZ**. ۲. **===** vs **==**؟
- بدون **coercion**. ۳. **undefined** vs **null**؟ **undefined** پیش‌فرض؛ **null** عمدی خالی. ۴. **this** چیست؟ مرجع شیء فراخوانی‌کننده؛ Arrow از بیرون.
۵. **Closure** چیست؟ تابع + محیط **lexical**. ۶. **Promise** چیست؟ نتیجه آینده.
۷. تفاوت **map** و **forEach**؟ **map** آرایه جدید؛ **forEach** void. ۸. **typeof null**؟ "object" – باگ تاریخی.
۹. **Event Bubbling**؟ انتشار از فرزند به والد. ۱۰. **use strict**؟ حالت سخت‌گیرانه.

۱۱. **Event Loop** ؟ `Stack → Microtask → Macrotask → Render`. ۱۲. چرا `[] == []` ؟ `!true` ؟ `[]` ؟ `"" → false → []` ؟
 ۱۳. `call, apply, bind` ؟ فوراً، `bind` تابع جدید. ۱۴. **Prototype Chain** ؟ زنجیره جستجو تا `false → 0 == 0`.
 ۱۵. **Map vs Object** ؟ `Map` کلید هر نوع، بدون `pollution`. ۱۶. **Debounce vs Throttle** ؟ `Debounce` بعد سکوت؛ `Throttle` هر `X ms`. ۱۷. **async/await** ؟ `Syntax sugar` روی `Promise`. ۱۸. چرا `forEach` با `async` نه؟
 ۱۹. **Shallow vs Deep** ؟ `Shallow` سطح اول؛ `Deep` همه — `structuredClone`. ۲۰. **WeakMap** ؟ کلیدهای ضعیف — مانع `GC` نمی‌شود. ۲۱. **Proxy** ؟ `Intercept` رفتار شیء. ۲۲. **Symbol** ؟ کلید یکتا. ۲۳.
 ۲۴. **slice vs splice** ؟ `slice` غیرجهشی، `splice` جهشی. ۲۵. **Immutable** ؟ چرا بهتر؟ ۲۶. **structuredClone vs JSON** ؟ اولی `Date, Map, circular` را هم `clone` می‌کند. ۲۷.
 ۲۸. **Object.groupBy** ؟ گروه‌بندی — `ES2024`. ۲۹. `at(-1)` ؟ اندیس منفی — `ES2022`. ۳۰. `import()` ؟ `code splitting` — `Dynamic import`.
 ۳۱. `undefined`.

۳۱. **Hidden Classes** ؟ بهینه‌سازی `V8`؛ ترتیب ویژگی مهم. ۳۲. **Deoptimization** ؟ دور ریختن کد بهینه وقتی فرض می‌شکند. ۳۳. **Monomorphic vs Polymorphic** ؟ `Monomorphic` یک نوع — سریع. ۳۴. **Micro vs Macro** ؟
`Micro` بعد هر `task`؛ `Macro` هر بار یکی. ۳۵. چرا `setTimeout(0)` بعد `Promise microtask`؟ ۳۶. **Memory Leak** ؟ `Global, Closure, EventListener, setInterval, Detached DOM`. ۳۷. **Prototype Pollution** ؟
 آلوده کردن `Object.prototype`. ۳۸. **XSS** ؟ تزریق اسکریپت از `innerHTML` ناامن. ۳۹. **CSP** ؟ `Header` منابع مجاز. ۴۰. **WeakRef vs WeakMap** ؟ `WeakRef` اشاره ضعیف؛ `WeakMap` دیکشنری با کلید ضعیف. ۴۱. **Decorator** ؟
`Wrap` کلاس/متد. ۴۲. **using** ؟ آزادسازی خودکار — `ES2024`. ۴۳. **any vs unknown** ؟ `any TS` خاموش؛ `unknown` امن. ۴۴. **satisfies** ؟ چک نوع بدون تحمیل — بهتر از `as`. ۴۵. **Tree Shaking** ؟ حذف کد استفاده‌نشده با `ESM`. ۴۶.
Composition vs Inheritance ؟ `Composition` انعطاف‌پذیرتر. ۴۷. **Observer** ؟ `Pub/Sub`. ۴۸. **race vs any** ؟ اولین `any`؛ اولین `settle`؛ اولین `fulfill`. ۴۹. **FinalizationRegistry** ؟ فهمیدن `GC`. ۵۰. چگونه `JS` را سریع‌تر کنیم؟
`Monomorphic`، `hidden class`، `for` به جای `forEach` در `Worker`، `hot path`.

✦ مصاحبه

برای Senior همیشه با مثال و trade-off جواب بده.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- یک مثال بازار کار که این مفهوم در آن حیاتی است؟

تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یکبار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی — مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

فارسی	انگلیسی	توضیح
کلژر	Closure	تابع + محیط lexical
زنجیره پروتوتایپ	Prototype Chain	زنجیره جستجو
بالا کشیدن	Hoisting	انتقال تعریف به ابتدای scope
ناحیه مرده زمانی	TDZ	فاصله شروع تا تعریف let/const
تک‌نخی	Single-threaded	یک Call Stack
حلقه رویداد	Event Loop	مدیریت Async
ریزکار	Microtask	Promise
درشت‌کار	Macrotask	setTimeout
قول	Promise	نتیجه آینده
انتظار	Await	توقف async
همزمانی	Concurrency	اجرای همپوشان
موازی	Parallelism	اجرای همزمان
جهشی	Mutating	تغییر اصلی
ناجهشی	Non-mutating	کپی جدید
ژرف‌کپی	Deep Clone	کپی همه سطوح
کم‌عمق‌کپی	Shallow Copy	کپی سطح اول
حافظه نهان	Cache	ذخیره نتیجه
یادسپاری	Memoization	کش نتیجه تابع
بازتاب	Reflect	API متا
پراکسی	Proxy	رهگیر رفتار
نماد	Symbol	کلید یکتا
تکرارپذیر	Iterable	دارای iterator
تکرارگر	Iterator	دارای next
مولد	Generator	تابع با yield

فارسی	انگلیسی	توضیح
تزئین کننده	Decorator	Wrap کلاس
ترکیب	Composition	ساخت از قطعات
ارث بری	Inheritance	is-a
مشاهده گر	Observer	Pub/Sub
کارخانه	Factory	سازنده شیء
یگانه	Singleton	یک نمونه
راهبرد	Strategy	انتخاب الگوریتم
آلودگی پروتوتایپ	Prototype Pollution	آلوده کردن prototype
حمله میان وبگاهی	XSS	تزریق اسکریپت
جعل درخواست	CSRF	درخواست با کوکی
سیاست امنیت محتوا	CSP	Header امنیتی
جمع آوری زباله	GC	آزادسازی حافظه
پشته	Stack	LIFO
هیپ	Heap	حافظه اشیا
کلاس پنهان	Hidden Class	بهینه سازی V8
بهینه زدایی	Deopt	دور ریختن کد بهینه
تک ریخت	Monomorphic	یک نوع – سریع
چند ریخت	Polymorphic	چند نوع – کند
درخت تکانی	Tree Shaking	حذف کد استفاده نشده
تقسیم کد	Code Splitting	Chunk جدا
بارگذاری تنبل	Lazy Loading	هنگام نیاز
تاخیر انداز	Debounce	بعد سکوت
گلوگاه گیر	Throttle	هر X ms
مرجع ضعیف	WeakRef	بدون جلوگیری GC

فارسی	انگلیسی	توضیح
نقشه ضعیف	WeakMap	کلید ضعیف
مجموعه	Set	بدون تکرار
نگاشت	Map	دیکشنری
ماژول ECMAScript	ESM	import/export
ماژول CommonJS	CJS	require
باندلر	Bundler	Vite, Webpack
پوشش	Coverage	درصد تست
خالص	Pure Function	بدون side effect
تغییرناپذیری	Immutability	عدم جهش
تابع درجه یک	First-Class	تابع به عنوان مقدار
کرینگ	Currying	$f(a,b) \rightarrow f(a)(b)$
ترکیب تابع	Composition	$f(g(x))$
مدیریت منبع	Resource Management	using

جمع بندی ≡

این واژه نامه را پرینت کن – در مصاحبه و Code Review به کارت می آید.

تمرین فعال – برای جا افتادن عمیق

تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده ای – دوباره بخوان.

تمرین ۲ – Active Recall

کتاب را ببند و این سوال ها را جواب بده:

مدل ذهنی این فصل چیست؟ آن را نقاشی کن.

- ♦ ۲ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

✓ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟
- ☐ می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

38 نقشه راه بعد از کتاب

حالا که JS را عمیق فهمیدی، وقت انتخاب مسیر است.

🎯 اهداف یادگیری این فصل

بعد از این فصل می‌توانی:

- ♦ مدل ذهنی اصلی فصل را روی کاغذ بکشی و برای یک Junior توضیح دهی
- ♦ ۳ اشتباه رایج این مبحث را در Code Review تشخیص دهی
- ♦ آن را در یک پروژه واقعی (با تست) پیاده‌سازی کنی
- ♦ در مصاحبه ۲، FAANG، دقیقه با مثال و trade-off توضیح دهی

🤔 چرا این فصل برای بازار کار حیاتی است؟

این فصل پایه‌ای است که در ۹۰٪ مصاحبه‌های Mid/Senior پرسیده می‌شود. شرکت‌های بزرگ به‌جای حفظ API، درک عمیق می‌خواهند. اگر این فصل را سطحی بخوانی، در فصل‌های بعد (Event Loop، Performance، Architecture) گیر می‌کنی.

i مدل ذهنی این فصل

قبل از خواندن کد، سعی کن یک تشبیه دنیای واقعی برای این مفهوم بسازی – مثلاً Closure = کوله‌پشتی، Prototype = زنجیر، Event Loop = آشپز تکنخی با پیشخوان سفارش‌ها.

مسیرهای پیشنهادی

مسیر	بعد از این کتاب	مهارت بعدی
Frontend Pro	React 19، Next.js 16	کتاب React 19 فارسی
Backend	Node.js 22، Fastify	کتاب Node.js
Fullstack	tRPC، Prisma	TypeScript عمیق
AI Engineer	Vercel AI SDK	Embeddings، RAG

پروژه نهایی پیشنهادی

یک Task Manager واقعی بساز:

Frontend: Vite + TypeScript + IndexedDB

Backend: Bun + Hono + SQLite

Features: Offline-first, Sync, Real-time WebSocket, Kanban

Quality: Vitest 90% coverage, Playwright E2E, Biome, CI/CD

```
final-stack.js

{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest --coverage",
    "lint": "biome check --write ./src",
    "e2e": "playwright test"
  }
}
```

عادت‌های Senior

یک مفهوم، یک بازسازی

Lighthouse CI با Performance Budget

Error Tracking: Sentry از روز اول

TypeDoc و Docs as Code: JSDoc

منابع

کتاب: You Don't Know JS Yet

سایت: MDN Web Docs

تمرین: Exercism JS Track

پرفورمنس: V8 Blog

الگوها: patterns.dev

♦ آخرین نکته

جاوااسکریپت را حفظ نکن – بفهم. وقتی Event Loop، V8، و Prototype را فهمیدی، هر فریم‌ورکی ۲ روزه قابل یادگیری است.

|| خدا حافظی

این کتاب را با عشق به جامعه فارسی‌زبان نوشتم. اگر مفید بود، با Star و اشتراک‌گذاری حمایت کن.

کدنویسی خوش! 🚀

JS

```
console.log("JavaScript is love – when you understand it deeply ❤️");
```

|| 🧪 تمرین فعال – برای جا افتادن عمیق

♦ تمرین ۱ – Feynman Technique

سعی کن مفهوم اصلی این فصل را به یک دوست غیرفنی در ۲ دقیقه توضیح دهی – بدون کلمات تخصصی. اگر گیر کردی، یعنی هنوز عمیق نفهمیده‌ای – دوباره بخوان.

♦ تمرین ۲ – Active Recall

کتاب را ببند و این سوال‌ها را جواب بده:

- ♦ مدل ذهنی این فصل چیست؟ آن را نقاشی کن.
- ♦ اشتباه رایج این مبحث چیست و راه‌حلش؟
- ♦ یک مثال بازار کار که این مفهوم در آن حیاتی است؟

♦ تمرین ۳ – چالش بازار کار

یک مینی‌پروژه کوچک بساز که این مفهوم را در عمل نشان دهد – با Vitest تست بنویس و در گیت‌هاب پوش کن. این رزومه تو می‌شود.

|| ✅ چک‌لیست فهم عمیق

- ☐ می‌توانی مدل ذهنی را روی کاغذ بکشی؟
- ☐ می‌توانی ۳ اشتباه رایج را نام ببری و درست کنی؟

- می‌توانی آن را در پروژه واقعی با تست پیاده کنی؟
- می‌توانی در مصاحبه ۲ دقیقه با trade-off توضیح دهی؟

✦ روش مطالعه مدرن – Spaced Repetition

این فصل را امروز بخوان، فردا ۱۰ دقیقه مرور کن، ۳ روز بعد دوباره، و ۷ روز بعد یک‌بار دیگر – با Anki. این روش ۳ برابر ماندگاری را بیشتر می‌کند.

🔗 اتصال به فصل بعد

فصل بعد بسط همین مدل ذهنی است – اگر این فصل را عمیق فهمیده باشی، فصل بعد فقط یک لایه جدید روی همین پایه است.

پایان مرجع

کدنویسی خوش! 🚀

اگر این کتاب برایتان مفید بود، آن را با توسعه‌دهندگان دیگر به اشتراک بگذارید.
جاوااسکریپت زبانی است که با فهم عمیقش، هر فریم‌ورکی برایتان ساده می‌شود.

JavaScript ES2025

TypeScript

Node.js 22

V8 Engine

Performance

گیت‌هاب `github.com/rezaian-dev`

مرجع React 19 `react-19-persian-guide`

گردآوری و تدوین حرفه‌ای · نسخه ۲۰۲۶ · ویرایش ۱.۰.۰